

A Major Project Report

On

**“A MEMORY BASED FFT PROCESSOR DESIGN WITH GENERALISED
EFFICIENT CONFLICT FREE ADDRESS SCHEMES”**

Submitted By

D.HITESH KUMAR 16K91A0473

P.PUSHPANJALI 16K91A0496

SAI VIKHILA 16K91A04A3

A Dissertation Submitted in Partial Fulfillment of the Requirement for the award of

Bachelor of Technology

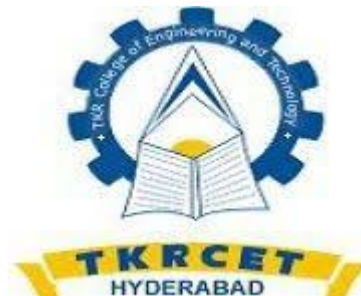
In

Electronics and Communication Engineering

Under the esteemed guidance of

Mrs. G. ANITHA CHOWDARY

Assoc. Professor, ECE Dept.



(2019-2020)

Department of Electronics and Communication Engineering

TKR COLLEGE OF ENGINEERING AND TECHNOLOGY

(Affiliated to JNTU Hyderabad, Accredited by NBA)

MEDBOWLI, MEERPET, SAROORNAGAR, HYDERABAD -500097

CERTIFICATE

This is to certify that the major project entitled “**A MEMORY BASED FFT PROCESSOR DESIGN WITH GENERALISED EFFICIENT CONFLICT FREE ADDRESS SCHEMES**” submitted by

D.HITESH KUMAR 16K91A0473

P.PUSHPANJALI 16K91A0496

SAI VIKHILA 16K91A04A3

is a record of bonafide work carried out by them during the academic year 2019-2020, under the guidance and supervision of

MR.A.THEJA

Assoc. Professor

Coordinator

MRS.J.SRAVANTHI

Asst. Professor

Coordinator

MRS.Y.PRIYANKA

Assoc. Professor

Coordinator

Mrs. G. ANITHA CHOWDARY

Assoc.Professor

Internal guide

Dr. D. NAGESHWAR RAO

Head of the Department ECE

External Examiner:

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of the task would be great but incomplete without mentioning the people who made it possible with their constant guidance.

We would like to express our gratitude to all people behind the screen who helped us to transform an idea into the real application.

We would like to thank our internal guide **Mrs. G. ANITHA CHOWDARY**, Associate Professor of department, ECE for guiding and correcting various documents with lot of attention and care.

We are grateful to our project coordinators **Mr. A. THEJA**, Associate Professor, **Mrs. Y. PRIYANKA**, Associate Professor and **Mrs. J. SRAVANTHI**, Assistant Professor of department for Electronics and Communication Engineering, for their complete coordination throughout our project.

We are profoundly thanking Dr. **D. NAGESHWAR RAO**, Head of the department for Electronics and Communication Engineering who has been the source of inspiration and laid a great support to our work.

We are grateful to our principal **Dr. D. V. RAVI SHANKAR** for the conducive environment created by him in the college for effective completion of project undertaken by us.

We would also like to thank our faculty members without whom this major project would have been distant reality.

DECLARATION

We hereby declare that the Major project report entitled “**A MEMORY BASED FFT PROCESSOR DESIGN WITH GENERALISED EFFICIENT CONFLICT FREE ADDRESS SCHEMES**”, is original and bonafide work of our own for the award of **BACHELOR OF TECHNOLOGY** and submitted to the **DEPARTMENT of ECE, TKR COLLEGE OF ENGINEERING & TECH.**, Hyderabad under the guidance of **Mrs. G.ANITHA CHOWDARY**, Assoc. Professor and has not been copied from earlier reports.

By

D.HITESH KUMAR (16K91A0473)

P.PUSHPANJALI (16K91A0496)

SAI VIKHILA (16K91A04A3)

ABSTRACT

The fast Fourier transform (FFT) is a commonly used algorithm in digital signal processing areas, such as imaging applications and communication systems. Memory-based FFTs pass the data multiple times through a single butterfly processing element (PE) or set of PEs, with several memory banks to hold intermediate results, processing the data recursively regardless of computation length. Conflict-free address schemes for concurrent data access from different memory banks become an essential problem.

This paper presents the design and implementation of memory-based fast Fourier transform (FFT) processors with generalized efficient, conflict-free address schemes. We unified the conflict-free address schemes of three different FFT lengths, including the single-power points, the common nonsingle-power points, and the non-single-power points applied with a prime factor algorithm. Though the three cases differ in terms of decomposition, they are all compatible with memory-based architecture by the way of the proposed address schemes. Moreover, the decomposition algorithm utilizes a method, named high-radix–small-butterfly (HRSB), to decrease the computation cycles and eliminate the complexity of the processing engine

INDEX

S.No.	Name of the topic	Pg. No
1	VLSI	01
1.1	Introduction	01
1.2	History	02
1.3	Integrated circuit	03
1.4	Developments	04
1.5	Generations	05
1.5.1	ssi, msi, lsi&vlsi	05
1.5.2	ulsi, wsi, soc, 3d-ic	07
1.6	VLSI design flow	08
1.7	Emergence of hardware Description language	08
1.8	Hardware Description language	09
2	Verilog HDL	10
2.1	Introduction to Verilog	10
2.2	Verilog features	10
2.3	VLSI design flow	11
2.4	Design specification	11
2.5	RTL description	11
2.5.1	Coding styles	12
2.5.2	Functional verification & testing	12

S.No	Name of the topic	Pg. No
2.6	Logic synthesis	12
2.7	Floor planning automatic placeand route	13
2.7.1	Physical layout	13
2.8	Layout verification	13
2.9	Modules	13
2.10	Ports	14
2.11	Port declaration	15
2.12	Port connection rules	15
2.12.1	Inputs	15
2.12.2	Outputs	16
2.12.3	In outs	16
2.12.3.1	Ports connection to external signals	16
2.12.3.2	Port by order list	16
2.12.3.3	Port by name	17
3	Software development requirement Analysis	18
3.1	Xilinx ISE	18
3.2	Creating a new project and source	19
3.3	Creating a Verilog source	21
3.4	Using language templates (Verilog)	23
3.5	Design simulation	23
4	Fast Fourier Transform	27
4.1	Introduction	27
4.2	History of FFT	28

4.3	How much faster is the FFT?	33
4.4	Conflict free address schemes	33
4.5	FFT architecture	34
4.6	Programmable FFT specific processors	36
4.7	Algorithm-specific processors	37
4.7.1	Radix-2 multipath delay Commutator	38
4.7.2	Radix-2 single single-path delay feedback	40
4.7.3	8-point dif r2sdc FFT	40
4.7.4	Radix-4 multipath delay Commutator	41
4.7.5	Radix-4 single-path delay Commutator	42
4.7.6	Radix-4 single-path delay feedback	43
4.7.7	Radix-2 2 single-path delay Commutator	44
5	Proposed system architecture	46
5.1	Block diagram	46
5.2	Description of blockdiagram	46
5.3	Schematic diagram	48
	Flowchart	49
	Result	50
	Advantages	52
	Conclusion	53
	Bibliography	54

LIST OF FIGURES

Figure No.	Name	Pg. No
2.1	VLSI design flow	11
2.2	Module design	14
2.3	Port connections	15
2.4	Adder Verilog code	17
3.1	Xilinx ise 8.1 icon	19
3.2	Xilinx ISE window	20
3.3	New project wizard-device properties	20
3.4	New project wizard_create new source	20
3.5	ISE project navigator	21
3.6	Modules	22
3.7	Xilinx_Ise window	22
3.8	Initial timing and clock wizard_initial timing tab	25
3.9	Waveforms	26
3.10	Simulation results	26
4.1	General-purpose programmable DSP processor	35
4.2	FFT-specific processor PDSP16515A	36
4.3	8-point fully parallel FFT processor	37
4.4	8-point R2MDC FFT	39

Figure No.	Name	Pg. No
4.5	Radix-2 Single-path Delay Feedback	40
4.6	64-point DIF R4MDC FFT	41
4.7	16-point DIF Radix-4 Single-Path	
	Delay Commutator (R4SDC) FFT	42
4.8	64-point DIF R4SDF FFT	43
4.9	Radix-4 SDF butterfly	44
4.10	Radix-2 2 Single-path Delay Commutator	45
5.1	Block diagram of the project	46
5.2	Schematic diagram of the project	48
5.3	Flow chart	49
5.4	output waveform	50
5.5	power analyzer	51

CHAPTER-1

VLSI

1.1 INTRODUCTION

Digital systems are highly complex at their most detailed level. They may consist of millions of elements i.e., transistors or logic gates. For many decades, logic schematics served as the lingua franca of logic design, but not anymore. Today, hardware complexity has grown to such a degree that a schematic with logic gates is almost useless as it shows only a web of connectivity and not functionality of design. Since the 1970s, computer engineers, electrical engineers and electronics engineers have moved toward Hardware description language (HDLs).

Digital circuit has rapidly evolved over the last twenty five years. The earliest digital circuits were designed with vacuum tubes and transistors. Integrated circuits were then invented where logic gates were placed on a single chip. The first IC chip was small scale integration (SSI) chips where the gate count is small. When technology became sophisticated, designers were able to place circuits with hundreds of gates on a chip. These chips were called MSI chips with advent of LSI; designers could put thousands of gates on a single chip. At this point, design process is getting complicated and designers felt the need to automate these processes.

With the advent of VLSI technology, designers could design single chip with more than hundred thousand gates. Because of the complexity of these circuits computer aided techniques became critical for verification and for designing these digital circuits.

One way to lead with increasing complexity of electronic systems and the increasing time to market is to design at high levels of abstraction. Traditional paper and pencil and capture and simulate methods have largely given way to the described UN synthesized approach.

For these reasons, hardware description languages have played an important role in describe and synthesis design methodology. They are used for specification, simulation and synthesis of an electronic system. This helps to reduce the complexity in designing and products are made to be available in market quickly.

The components of a digital system can be classified as being specific to an application or as being standard circuits. Standard components are taken from a set that has been used in other systems. MSI components are standard circuits and their use results in a significant reduction in the total cost as compared to the cost of using SSI Circuits. In contrast, specific components are particular to the system being implemented and are not commonly found among the standard components.

The implementation of specific circuits with LSI chips can be done by means of IC that can be programmed to provide the required logic.

1.2 HISTORY

The history of the transistor dates to the 1920s when several inventors attempted devices that were intended to control current in solid-state diodes and convert them into triodes. Success did not come until after WWII, during which the attempt to improve silicon and germanium crystals for use as radar detectors led to improvements in fabrication and in the understanding of quantum mechanical states of carriers in semiconductors. Then scientists who had been diverted to radar development returned to solid-state device development. With the invention of transistors at Bell Labs in 1947, the field of electronics shifted from vacuum tubes to solid-state devices. With the small transistor at their hands, electrical engineers of the 1950s saw the possibilities of constructing far more advanced circuits. As the complexity of circuits grew, problems arose. One problem was the size of the circuit. A complex circuit, like a computer, was dependent on speed. If the components of the computer were too large or the wires interconnecting them too long, the electric signals could not travel fast enough through the circuit, thus making the computer too slow to be effective.

The invention of the integrated circuit by Jack Kilby and Robert Noyce solved this problem by making all the components and the chip out of the same block (monolith) of semiconductor material. Although the first integrated circuit was crude and had some problems, the idea was ground breaking. By making all the parts out of the same block of material and adding the metal needed to connect them as a layer on top of it, there was no need for discrete components. No more wires and components had to be assembled manually. The circuits could be made smaller, and the manufacturing process could be automated. From here, the idea of integrating all components on

a single silicon wafer came into existence, which led to development in small-scale integration (SSI) in the early 1960s, medium-scale integration (MSI) in the late 1960s, and then large-scale integration (LSI) as well as VLSI in the 1970s and 1980s, with tens of thousands of transistors on a single chip (later hundreds of thousands, then millions, and now billions (10⁹)).

1.3 INTEGRATED CIRCUIT

An integrated circuit or monolithic integrated circuit (also referred to as an IC, a chip, or a microchip) is a set of electronic circuits on one small plate ("chip") of semiconductor material, normally silicon. This can be made much smaller than a discrete circuit made from independent electronic components. ICs can be made very compact, having up to several billion transistors and other electronic components in an area the size of a fingernail. The width of each conducting line in a circuit can be made smaller and smaller as the technology advances, in 2008 it dropped below 100 nanometers, and now is tens of nanometers.

ICs were made possible by experimental discoveries showing that semiconductor devices could perform the functions of vacuum tubes and by mid-20th-century technology advancements in semiconductor device fabrication. The integration of large numbers of tiny transistors into a small chip was an enormous improvement over the manual assembly of circuits using discrete electronic components.

The integrated circuit's mass production capability, reliability and building-block approach to circuit design ensured the rapid adoption of standardized integrated circuits in place of designs using discrete transistors.

ICs have two main advantages over discrete circuits: cost and performance. Cost is low because the chips, with all their components, are printed as a unit by photolithography rather than being constructed one transistor at a time. Furthermore, packaged ICs use much less material than discrete circuits. Performance is high because the IC's components switch quickly and consume little power (compared to their discrete counterparts) as a result of the small size and close proximity of the components. As of 2012, typical chip areas range from a few square millimeters to around 450 mm², with up to 9 million transistors per mm².

Integrated circuits are used in virtually all electronic equipment today and have revolutionized the world of electronics. Computers, mobile phones, and other digital home appliances are now

inextricable parts of the structure of modern societies, made possible by the low cost of integrated circuits.

tiny transistors into a small chip was an enormous improvement over the manual assembly of circuits using discrete electronic components.

The integrated circuit's mass production capability, reliability and building-block approach to circuit design ensured the rapid adoption of standardized integrated circuits in place of designs using discrete transistors.

ICs have two main advantages over discrete circuits: cost and performance. Cost is low because the chips, with all their components, are printed as a unit by photolithography rather than being constructed one transistor at a time. Furthermore, packaged ICs use much less material than discrete circuits. Performance is high because the IC's components switch quickly and consume little power (compared to their discrete counterparts) as a result of the small size and close proximity of the components. As of 2012, typical chip areas range from a few square millimetres to around 450 mm², with up to 9 million transistors per mm².

Integrated circuits are used in virtually all electronic equipment today and have revolutionized the world of electronics. Computers, mobile phones, and other digital home appliances are now inextricable parts of the structure of modern societies, made possible by the low cost of integrated circuits.

1.4 DEVELOPMENTS

The first semiconductor chips held two transistors each. Subsequent advances added more transistors, and as a consequence, more individual functions or systems were integrated over time. The first integrated circuits held only a few devices, perhaps as many as ten diodes, transistors, resistors, and capacitors, making it possible to fabricate one or more logic gates on a single device. Now known retrospectively as small-scale integration (SSI), improvements in technique led to devices with hundreds of logic gates, known as medium-scale integration (MSI). Further improvements led to large-scale integration (LSI) i.e. systems with at least a thousand logic gates. Current technology has moved far past this mark and today's microprocessors have many millions of gates and billions of individual transistors.

At one time, there was an effort to name and calibrate various levels of large-scale integration above VLSI. Terms like ultra-large-scale integration (ULSI) were used. But the huge number of gates and transistors available on common devices has rendered such fine distinctions moot. Terms suggesting greater than VLSI levels of integration are no longer in widespread use.

As of early 2008, billion-transistor processors are commercially available. This became more commonplace as semiconductor fabrication advanced from the then-current generation of 65 nm processes. Current designs, unlike the earliest devices, use extensive design automation and automated logic synthesis to lay out the transistors, enabling higher levels of complexity in the resulting logic functionality. Certain high-performance logic blocks like the SRAM (static random-access memory) cell, are still designed by hand to ensure the highest efficiency. VLSI technology may be moving toward further radical miniaturization with introduction of NEMS technology.

1.5 GENERATIONS

In the early days of simple integrated circuits, the technology's large scale limited each chip to only a few transistors, and the low degree of integration meant the design process was relatively simple. Manufacturing yields were also quite low by today's standards. As the technology progressed, millions, then billions of transistors could be placed on one chip, and good designs required thorough planning, giving rise to new design methods.

1.5.1 SSI, MSI, LSI & VLSI

The first integrated circuits contained only a few transistors. Called "small-scale integration" (SSI), digital circuits containing transistors numbering in the tens provided a few logic gates for example, while early linear ICs such as the Plessey SL201 or the Philips TAA320 had as few as two transistors. The term Large Scale Integration was first used by IBM scientist Rolf Landauer when describing the theoretical concept from there came the terms for SSI, MSI, VLSI, and ULSI.

SSI circuits were crucial to early aerospace projects, and aerospace projects helped inspire development of the technology. Both the Minuteman missile and Apollo program needed lightweight digital computers for their inertial guidance systems; the Apollo guidance computer

led and motivated the integrated-circuit technology, while the Minuteman missile forced it into mass-production. The Minuteman missile program and various other Navy programs accounted for the total \$4 million integrated circuit market in 1962, and by 1968, U.S. Government space and defence spending still accounted for 37% of the \$312 million total production. The demand by the U.S. Government supported the nascent integrated circuit market until costs fell enough to allow firms to penetrate the industrial and eventually the consumer markets. The average price per integrated circuit dropped from \$50.00 in 1962 to \$2.33 in 1968. Integrated circuits began to appear in consumer products by the turn of the decade, a typical application being FM inter-carrier sound processing in television receivers.

The next step in the development of integrated circuits, taken in the late 1960s, introduced devices which contained hundreds of transistors on each chip, called "medium-scale integration" (MSI). They were attractive economically because while they cost little more to produce than SSI devices, they allowed more complex systems to be produced using smaller circuit boards, less assembly work (because of fewer separate components), and a number of other advantages.

Further development, driven by the same economic factors, led to "large-scale integration" (LSI) in the mid-1970s, with tens of thousands of transistors per chip. Integrated circuits such as 1K-bit RAMs, calculator chips, and the first microprocessors, that began to be manufactured in moderate quantities in the early 1970s, had under 4000 transistors. True LSI circuits, approaching 10,000 transistors, began to be produced around 1974, for computer main memories and second-generation microprocessors.

The final step in the development process, starting in the 1980s and continuing through the present, was "very large-scale integration" (VLSI). The development started with hundreds of thousands of transistors in the early 1980s, and continues beyond several billion transistors as of 2009.

Multiple developments were required to achieve this increased density. Manufacturers moved to smaller design rules and cleaner fabrication facilities, so that they could make chips with more transistors and maintain adequate yield. The path of process improvements was summarized by the International Technology Roadmap for Semiconductors (ITRS). Design tools improved

enough to make it practical to finish these designs in a reasonable time. The more energy efficient CMOS replaced NMOS and PMOS, avoiding a prohibitive increase in power consumption.

In 1986 the first one megabit RAM chips were introduced, containing more than one million transistors. Microprocessor chips passed the million transistor mark in 1989 and the billion transistor mark in 2005. The trend continues largely unabated, with chips introduced in 2007 containing tens of billions of memory transistors.

1.5.2 ULSI, WSI, SOC, 3D-IC

To reflect further growth of the complexity, the term ULSI that stands for "ultra-large-scale integration" was proposed for chips of more than 1 million transistors.

Wafer-scale integration (WSI) is a means of building very large integrated circuits that uses an entire silicon wafer to produce a single "super-chip". Through a combination of large size and reduced packaging, WSI could lead to dramatically reduced costs for some systems, notably massively parallel supercomputers. The name is taken from the term Very-Large-Scale Integration, the current state of the art when WSI was being developed.

A system-on-a-chip (SoC or SOC) is an integrated circuit in which all the components needed for a computer or other system are included on a single chip. The design of such a device can be complex and costly, and building disparate components on a single piece of silicon may compromise the efficiency of some elements. However, these drawbacks are offset by lower manufacturing and assembly costs and by a greatly reduced power budget: because signals among the components are kept on-die, much less power is required (see Packaging).

A three-dimensional integrated circuit (3D-IC) has two or more layers of active electronic components that are integrated both vertically and horizontally into a single circuit. Communication between layers uses on-die signalling, so power consumption is much lower than in equivalent separate circuits. Judicious use of short vertical wires can substantially reduce overall wire length for faster operation

1.6 VLSI DESIGN FLOW

Typical design flow for designing VLSI circuits is shown in the tool flow diagram. This design flow is typically used by designers who use HDLs. In any design, specification is first. Specification describes the functionality, interface and overall architecture of the digital circuit to be designed. At this point, architects need not think about how they will implement their circuit. A behavioral description is then created to analyze the design in terms of functionality, performances and other high level issues. The behavioral description is manually converted to an RTL (Register Transfer Level) description in an HDL. The designer has to describe the data flow that will implement the desired digital circuit. From this point onward, the design process is done with assistance of CAD tools.

Logic synthesis tools convert the RTL description to a gate level net list. A gate level net list is a description of the circuit in terms of gates and connections between them. The gate level net list is input to an automatic place and route tool, which creates a layout. The layout is verified and then fabricated on a chip. Thus most digital design activity is concentrated on manually optimizing the RTL description of the circuit. After the RTL description is frozen, CAD tools are available to assist the designer in further process. Designing at RTL level has shrunk design cycle times from years to a few months.

1.7 EMERGENCE OF HARDWARE DESCRIPTION LANGUAGE

As designs got larger and complex, logic simulation assumed an important role in design process. For a long time, programming languages such as Fortran, pascal & c were been used to describe the computer programs that were been used to describe the computer programs that were sequential in nature. Similarly in digital design field, designers felt the need for a standard language to describe digital circuits. Thus HDL is came in to existence. HDLs allowed the designers to model the concurrency of processes found in hardware elements. HDLs such as VERILOG HDL & VHDL (Very high speed integrated circuit hardware description language).

1.8HARDWARE DESCRIPTION LANGUAGE

Two things distinguish an HDL from a linear language like “C”: Concurrency:

- The ability to do several things simultaneously i.e. different code-blocks can run concurrently.

Timing:

- Ability to represent the passing of time and sequence events accordingly.

CHAPTER 2

VERILOG HDL

2.1 INTRODUCTION TO VERILOG

- Verilog HDL is a Hardware Description Language (HDL).
- A Hardware Description Language is a language used to describe a digital system; one may describe a digital system at several levels.
- An HDL might describe the layout of the wires, resistors and transistors on an Integrated Circuit (IC) chip, i.e., the switch level.
- It might describe the logical gates and flip flops in a digital system, i.e., the gate level.
- An even higher level describes the registers and the transfers of vectors of information between registers. This is called the Register Transfer Level (RTL).
- Verilog supports all of these levels.
- A powerful feature of the Verilog HDL is that you can use the same language for describing, testing and debugging your system.

2.2 VERILOG FEATURES

- Strong Background: Supported by open verilog international and Institute of Electrical and Electronics Engineering standardized.
- Industrial support: Simulation is very fast and synthesis is very efficient.
- Universal: Entire process is allowed in one design environment.
- Extensibility: It also allows Verilog PLI for extension of Verilog capabilities

2.3 VLSI DESIGN FLOW

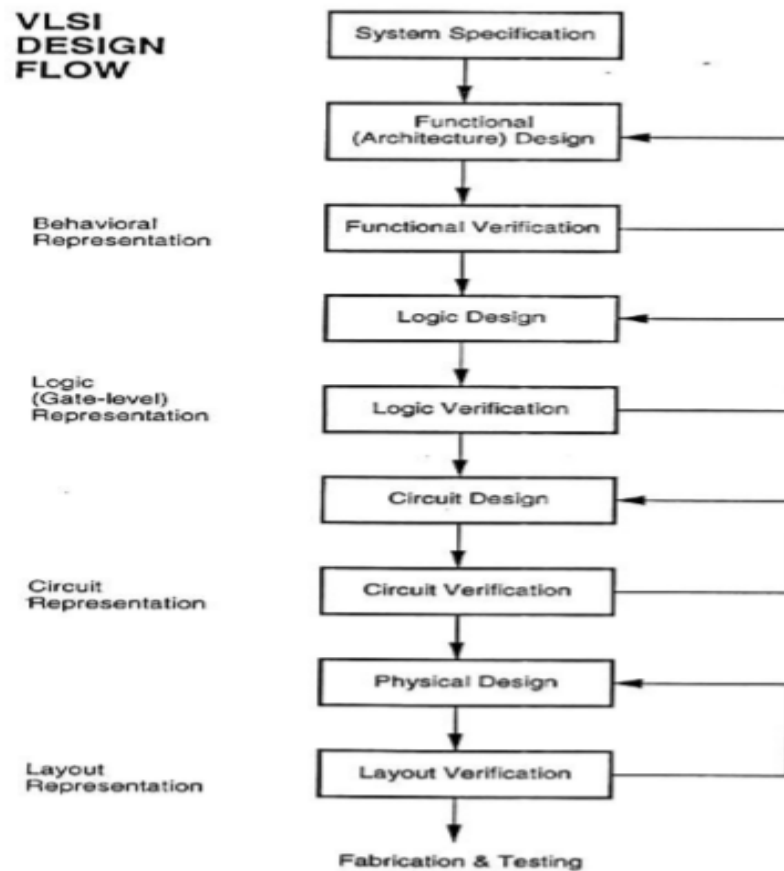


FIG 2.1: Vlsi design flow

2.4 DESIGN SPECIFICATION

- The project Specifications and requirements are written first.
- The digital circuit functionality is explained for the architecture to be designed.
- Specification: It uses wave former, test bench or word for drawing waveform.

2.5 RTL DESCRIPTION

- CAD Tools are used for coding format for the Conversation of Specification.

2.5.1 Coding Styles:

- Gate Level Modeling
- Data Flow Modeling
- Behavioral Modeling

2.5.2 Functional Verification & Testing

- The method of coding with respective inputs and outputs are going to be tested.
- Check the RTL Description once again if testing fails.
- Simulation: Using Xilinx, Verilog-XL, ModelSim,

2.6 LOGIC SYNTHESIS

- RTL description into Gate level -Net list form conservation.
- The circuit is described as a function of gates and connections.
- Synthesis: Synthesis is done by Altera and Xilinx, Simplify Pro, Leonardo Spectrum Design Compiler, FPGA Compiler.

Logical Verification and Testing

- Simulation and synthesis are used for functional Checking of HDL coding. Check the RTL description if fails.

2.7 FLOOR PLANNING AUTOMATIC PLACE AND ROUTE

- Layout is created with the respective gate level Net list.
- The blocks of the net list are arranged on the chip.
- Place & Route: Implement FPGA vendors P&R tool for FPGA. Very costly P&R tools like Apollo required for ASIC tools.

2.7.1 Physical Layout

- The process of describing a circuit description into the physical layout is called the Physical design, it explains the interconnections between the cells and routes position.

2.8 LAYOUT VERIFICATION

- Under layout verification first the physical layout structure has to be verified.
- Floor Planning Automatic Place and Route and RTL Description can be done for any modifications.

Implementation

- The design process is the final stage in implementation.
- Coding and RTL can be Implemented using Integrated circuits.

2.9MODULES

Distinct parts of a Verilog module consists of are as shown in below figure. The keyword module is the beginning of a module definition. In a module definition the module name, port list, port declarations, and optional parameters must come first in its definition. If the module has any ports to interact with the external environment then only Port list and port declarations are present. There are five components within a module

- Variable declarations,
- Dataflow statements
- Instantiation of lower modules
- Behavioral blocks
- Tasks or functions.

The following components may appear in any order and at any place in a given module definition.

The end module statement must always come last in a module definition. All components except module, module name, and end module are optional and can be mixed and matched as per

design needs. Multiple modules definition in a single file are allowed by Verilog. In the file the modules can be defined in any order.

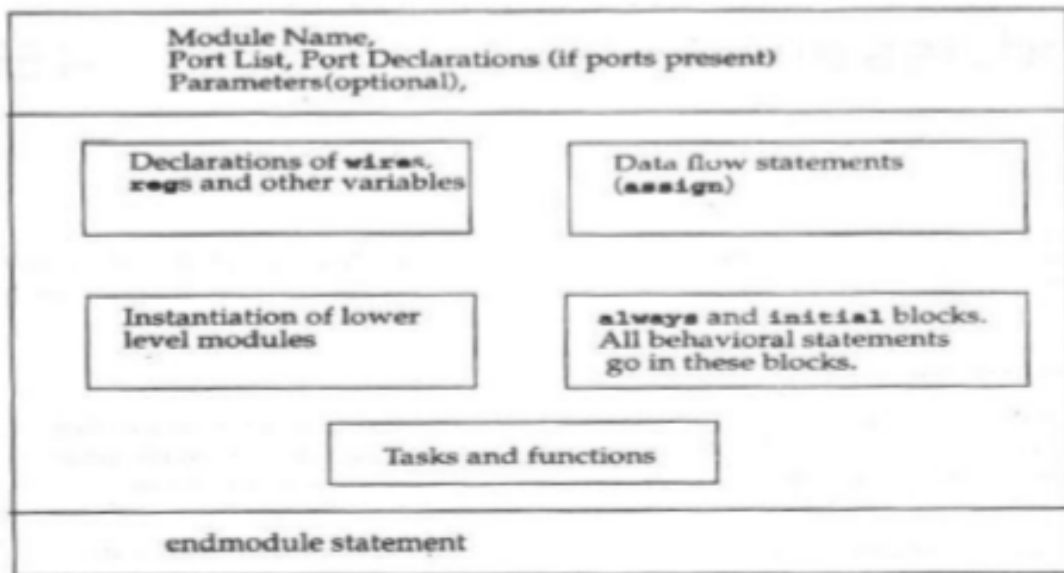


Fig 2.2: Module design

Example Module Structure:

```
module<module name>(<module_terminals_list>);
```

```
..... <module internals> .....
```

```
Endmodule
```

2.10 PORTS

The interface between a module and its environment is provided by the ports. The input/output pins of an Integrated Circuit chip are its ports. The environment cannot see the internals of the module. It is a great advantage for the designer. As long as the interface is not modified the internals of the module may be Modified without affecting its environment. Terminals are the synonyms for the ports.

2.11 PORT DECLARATION

The module may contain the declaration of all ports in the given list of ports. The declaration of ports is explained in detail below.

Verilog Keyword Type of Port

Input:- Input port

Output:- Output port

in out:-Bidirectional port

Depending on the direction of the port signal, each port in the port list is given a label as follows input, output, or in out.

2.12 PORT CONNECTION RULES

A port consisting of two units, primary unit is into the module and secondary unit is out of the module. The primary and secondary units are connected. When modules are instantiated within other modules there are rules governing port connections within module. If any port connection rules are violated then the Verilog simulator complains. The figure 5.6 shows the port connection rules.

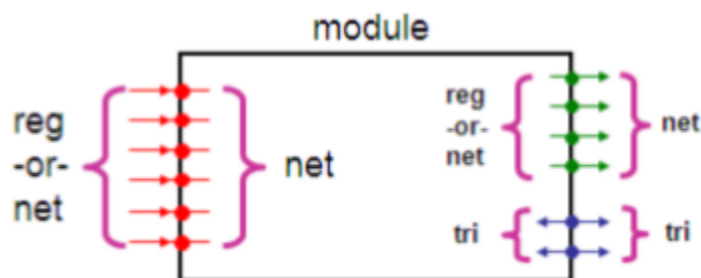


Fig 2.3: Port connections

2.12.1 INPUTS

- Internally must be of net data type (e.g. wire)
- Externally the inputs may be connected to a reg or net data type

2.12.2 OUTPUTS

- Internally may be of net or reg data type
- Externally must be connected to a net data type

2.12.3 INOUTS

- Internally must be of net data type (tri recommended)
- Externally must be connected to a net data type (tri recommended)

2.12.3.1 Ports Connection to External Signals:

Signals and Ports in a module can be connected in two ways. In the module definition those two methods cannot be mixed.

- Port by order list
- Port by name

2.12.3.2 Port by order list:

Most spontaneous method for learners is the Connecting port by order list. The order in which the ports in the ports list in the module definition must be connected in the same order.

Syntax for instantiation with port order list:

module name instance name (signal, signal...);

The external signals a, b, out appear in exactly the same order as the ports a, b, out in the module defined in adder in the below example.

```
module adder(a,b,out):
    input[1:0]a;
    input[1:0]b;
    output[1:0]out;
    wire[1:0]out;

    assign out=a+b;
endmodule
```

```
module top_example;
    reg[1:0]a;
    reg[1:0]b;
    wire[1:0]out;

    adder_ext(a,b,out);
endmodule
```

Fig 2.4:Adder Verilog code

2.12.3.3Port by name

For larger designs where the module have say 30 ports, it is almost impractical and possibility of errors if remembering the order of the ports in the module definition. There is capability to connect external signals to ports by the port names, rather than by position provided by the Verilog. Syntax for instantiation with port name:

Module name instance name (.port name (signal), .port name (signal)...);

The port connections in any order as long as the port name in the module definition correctly matches the external signal.

CHAPTER 3

SOFTWARE DEVELOPMENT REQUIREMENT ANALYSIS

3.1 XILINX ISE

Verify that Enable Enhanced Design Summary is selected. Create a New Project Create a new ISE project which will target the FPGA device on the Spartan-3 Startup Kit demo board. To create a new project: 1. Select File > New Project... The New Project Wizard appears. 2. Type tutorial in the Project Name field. 3. Enter or browse to a location (directory path) for the new project. A tutorial subdirectory is created automatically. 4. Verify that HDL is selected from the Top-Level Source Type list. 5. Click Next to move to the device properties page. 6. Fill in the properties in the table as shown below:

- Product Category: All
- Family: Spartan3
- Device: XC3S200
- Package: FT256
- Speed Grade: -4
- Top-Level Source Type: HDL
- Synthesis Tool: XST (VHDL/Verilog)
- Simulator: ISE Simulator (VHDL/Verilog)
- Preferred Language: Verilog (or VHDL)

Leave the default values in the remaining fields. When the table is complete, your project properties will look like the following:

3.2 CREATING A NEW PROJECT AND SOURCE

1. Start the Xilinx ISE 8.1i project navigator by double clicking the Xilinx ISE 8.1i icon on your desktop.



Fig 3.1: Xilinx ise 8.1 icon

2. Click on File and select New Project

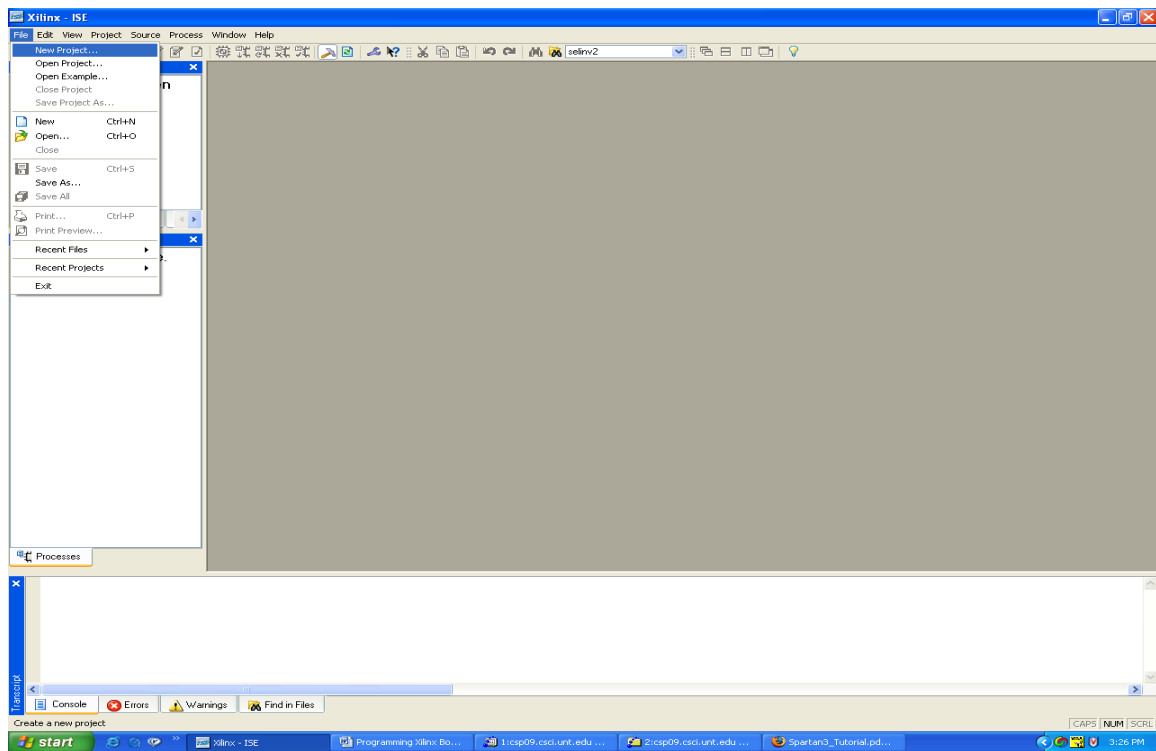


Fig 3.2: Xilinx ISE window

3. Select a project location and type the name you would like to call your project counter

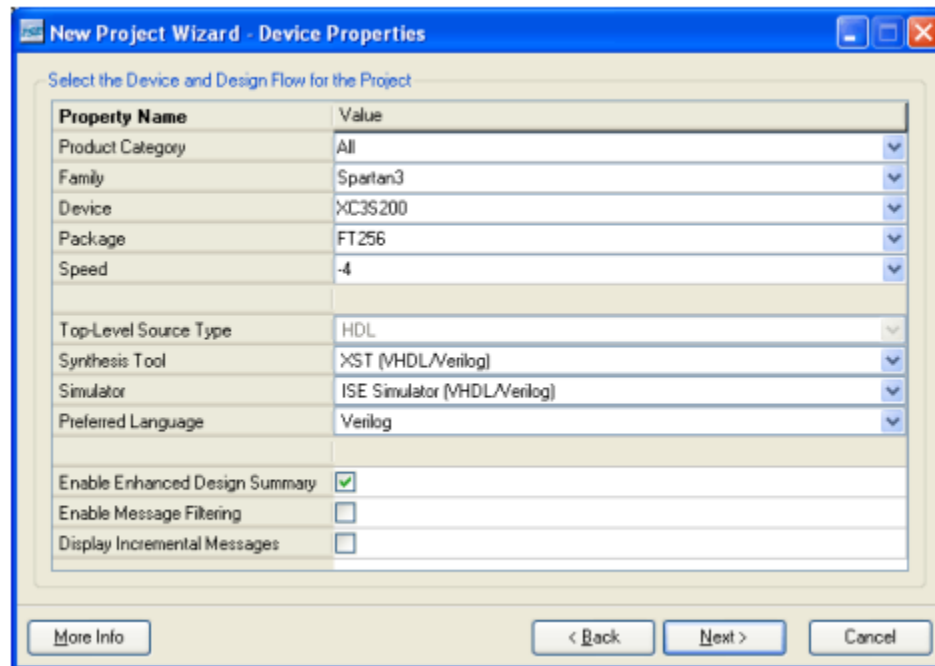


Fig 3.3: New project wizard-device properties

4. Click next

5. Click New Source

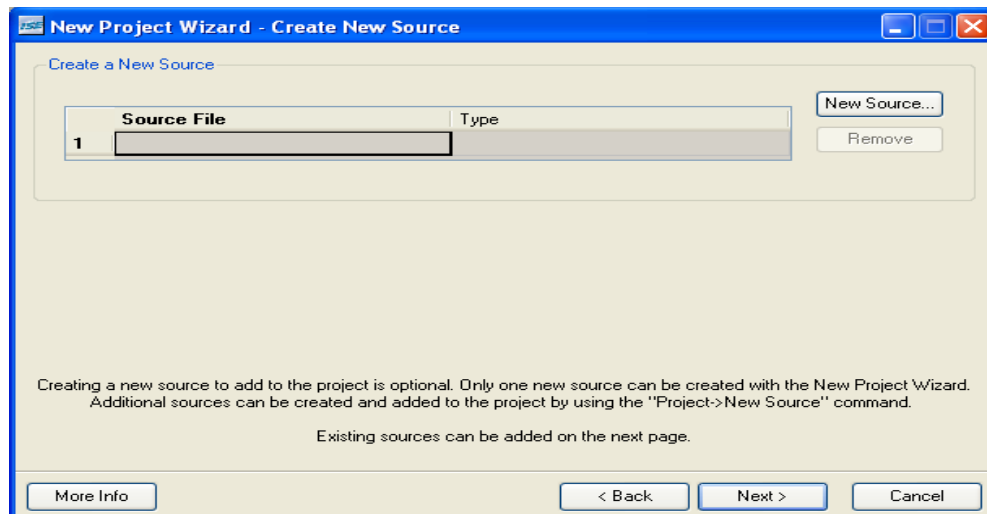


Fig 3.4: New project wizard_create new source

6. Click next

Select Verilog Module in the New Source Wizard window

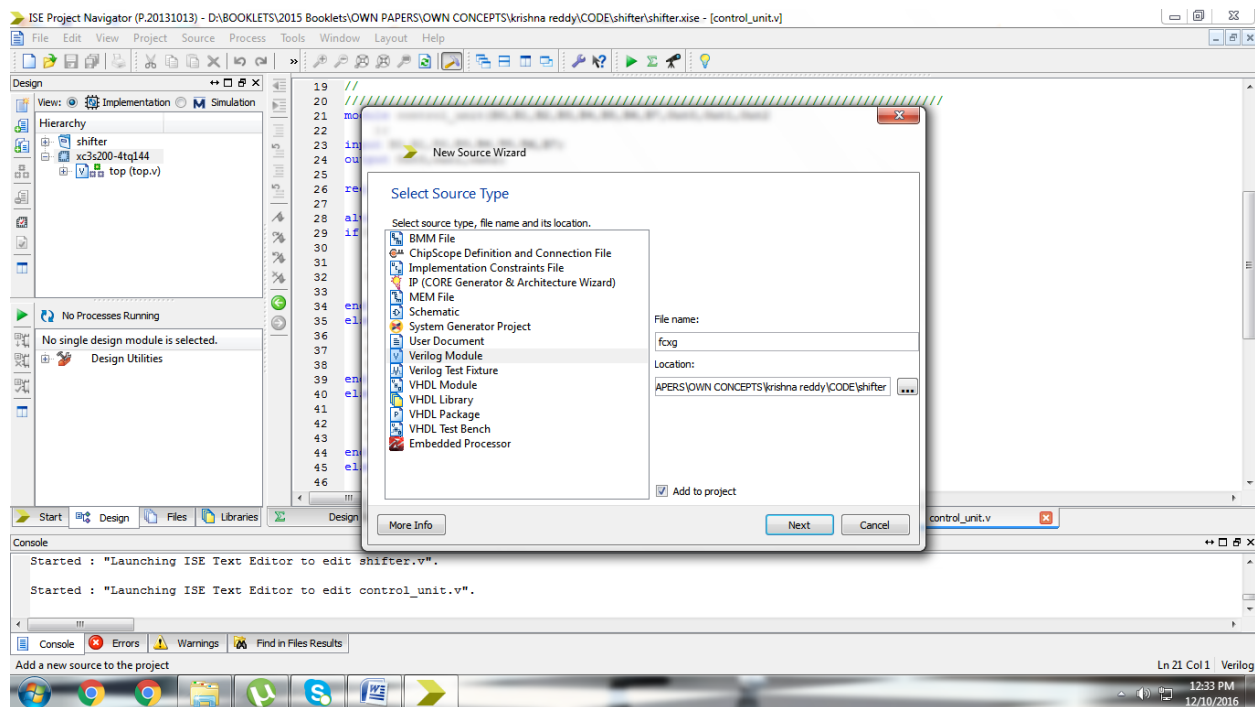


Fig 3.5: ISE project navigator

3.3 CREATING A VERILOG SOURCE

Create the top-level Verilog source file for the project as follows:

1. Click New Source in the New Project dialog box.
2. Select Verilog Module as the source type in the New Source dialog box.
3. Type in the file name counter.
4. Verify that the Add to Project checkbox is selected.
5. Click Next.
6. Declare the ports for the counter design by filling in the port information as shown below:

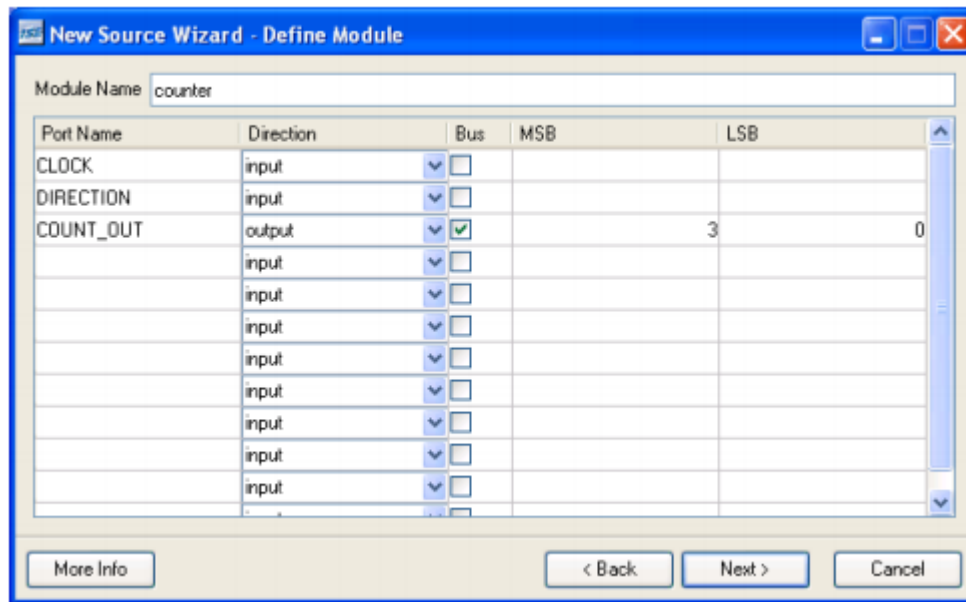


Fig 3.6: Modules

7. click next.

8. The source file containing the counter module displays in the Workspace, and the counter displays in the Sources tab, as shown below:

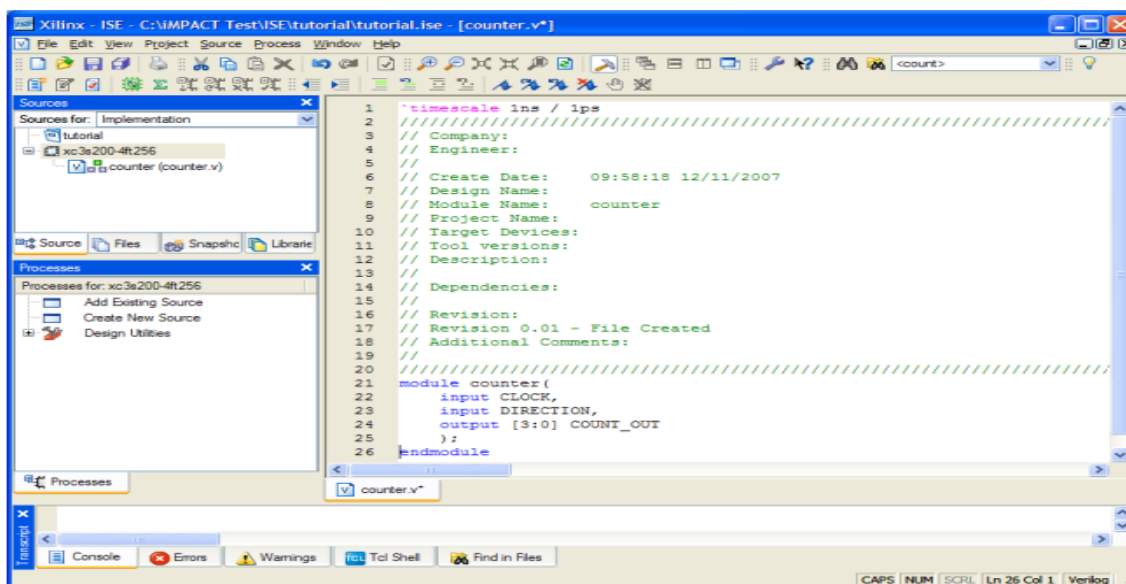


Fig 3.7: Xilinx_Ise window

3.4 USING LANGUAGE TEMPLATES (VERILOG)

The next step in creating the new source is to add the behavioral description for counter. Use a simple counter code example from the ISE Language Templates and customize it for the counter design.

1. Place the cursor on the line below the output [3:0] COUNT_OUT; statement.
2. Open the Language Templates by selecting Edit → Language Templates... Note: You can tile the Language Templates and the counter file by selecting Window → Tile Vertically to make them both visible.
3. Using the “+” symbol, browse to the following code example: Verilog → Synthesis Constructs → Coding Examples → Counters → Binary → Up/Down Counters → Simple Counter.
4. With Simple Counter selected, select Edit → Use in File, or select the Use Template in File toolbar button. This step copies the template into the counter source file.
5. Close the Language Templates

3.5 DESIGN SIMULATION

Verifying Functionality using Behavioral Simulation Create a test bench waveform containing input stimulus you can use to verify the functionality of the counter module. The test bench waveform is a graphical view of a test bench. Create the test bench waveform as follows:

1. Select the counter HDL file in the Sources window.
2. Create a new test bench source by selecting Project → New Source.
3. In the New Source Wizard, select Test Bench Waveform as the source type, and type counter_tbw in the File Name field.
4. Click Next.
5. The Associated Source page shows that you are associating the test bench waveform with the source file counter. Click Next.

6. The Summary page shows that the source will be added to the project, and it displays the source directory, type, and name. Click Finish.

7. You need to set the clock frequency, setup time and output delay times in the Initialize Timing dialog box before the test bench waveform editing window opens. The requirements for this design are the following:

- The counter must operate correctly with an input clock frequency = 25 MHz
- The DIRECTION input will be valid 10 ns before the rising edge of CLOCK
- The output (COUNT_OUT) must be valid 10 ns after the rising edge of CLOCK. The design requirements correspond with the values below. Fill in the fields in the Initialize Timing dialog box with the following information:
 - Clock High Time: 20 ns.
 - Clock Low Time: 20 ns.
 - Input Setup Time: 10 ns.
 - Output Valid Delay: 10 ns. ♦ Offset: 0 ns.
 - Global Signals: GSR (FPGA) Note: When GSR (FPGA) is enabled, 100 ns is added to the Offset value automatically.
- Initial Length of Test Bench: 1500 ns.

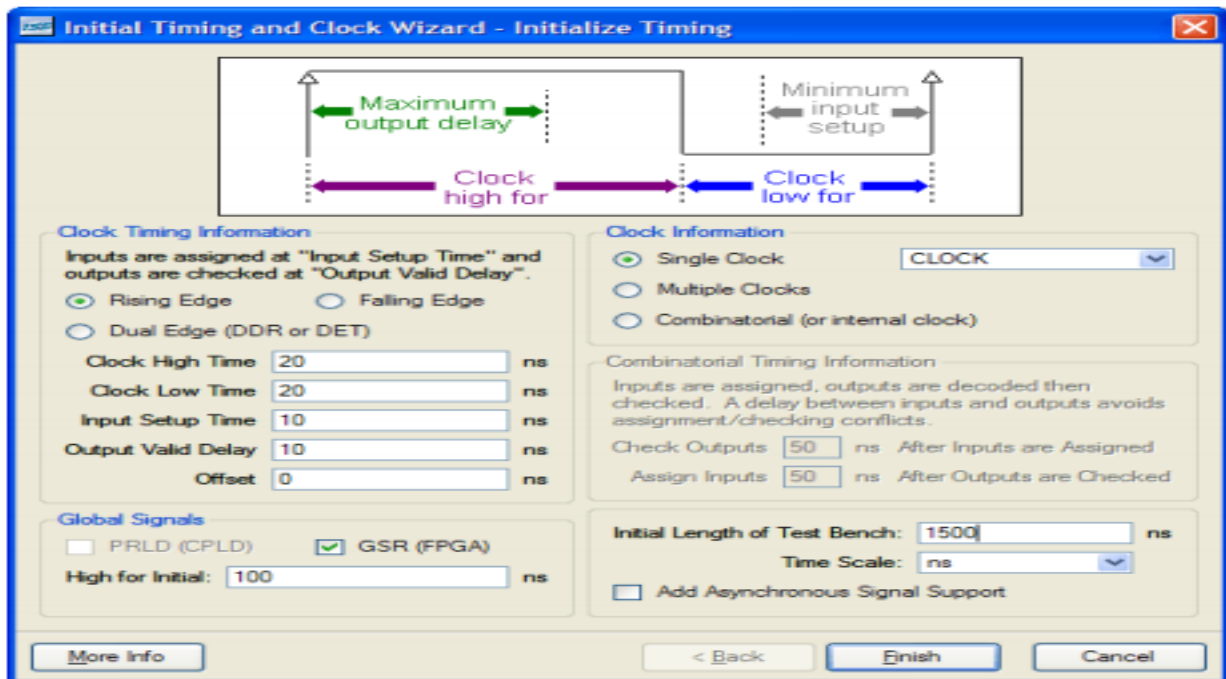


Fig 3.8: Initial timing and clock wizard_initial timing tab

8. Click Finish to complete the timing initialization.

9. The blue shaded areas that precede the rising edge of the CLOCK correspond to the Input Setup Time in the Initialize Timing dialog box. Toggle the DIRECTION port to define the input stimulus for the counter design as follows:

- Click on the blue cell at approximately the 300 ns to assert DIRECTION high so that the counter will count up.
- Click on the blue cell at approximately the 900 ns to assert DIRECTION low so that the counter will count down.

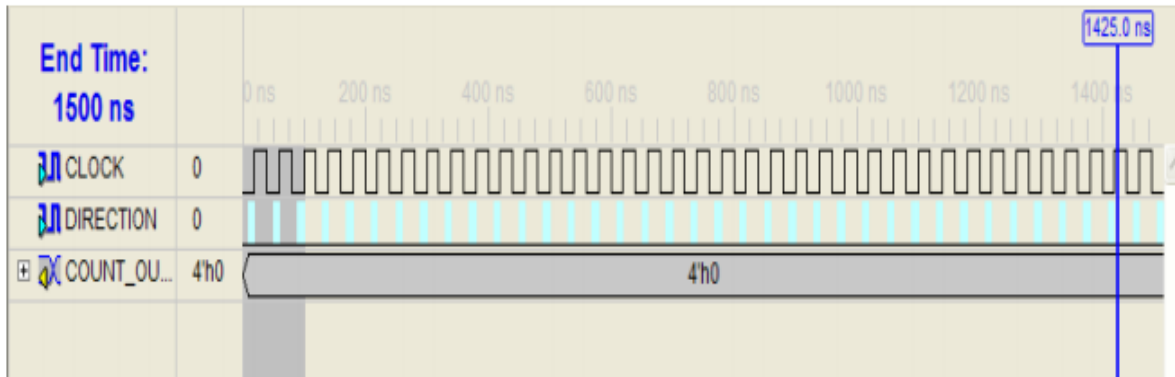


Fig 3.9: Waveforms

Simulation Results

decoder Project Status (02/18/2018 - 14:14:57)			
Project File:	adaptive.xise	Parser Errors:	No Errors
Module Name:	eliminator_block	Implementation State:	Synthesized
Target Device:	xc3s200-4tq144	• Errors:	No Errors
Product Version:	ISE 14.7	• Warnings:	4 Warnings (4 new)
Design Goal:	Balanced	• Routing Results:	
Design Strategy:	Xilinx Default (unlocked)	• Timing Constraints:	
Environment:	System Settings	• Final Timing Score:	

Device Utilization Summary (estimated values)				[-]
Logic Utilization	Used	Available	Utilization	
Number of Slices	4	1920	0%	
Number of Slice Flip Flops	8	3840	0%	
Number of 4 input LUTs	5	3840	0%	
Number of bonded IOBs	4	97	4%	
Number of GCLKs	1	8	12%	

Fig 3.10: Simulation results

CHAPTER 4

FAST FOURIER TRANSFORM

4.1 INTRODUCTION

The development of fast algorithms usually consists of using special properties of the algorithm of interest to remove redundant or unnecessary operations of a direct implementation. Because of the periodicity, symmetries, and orthogonality of the basic functions and the special relationship with convolution, the discrete Fourier transform (DFT) has enormous capacity for improvement of its arithmetic efficiency.

There are four main approaches to formulating efficient DFT [50] algorithms. The rest two break a DFT into multiple shorter ones. This is done in Multidimensional Index Mapping by using an index map and in Polynomial Description of Signals by polynomial reduction. The third is factoring the Signal Processing Operators which factors the DFT operator (matrix) into sparse factors. The DFT as Convolution or Filtering develops a method which converts a prime-length DFT into cyclic convolution. Still another approach is interesting where, for certain cases, the evaluation of the DFT can be posed recursively as evaluating a DFT in terms of two half-length DFTs which are each in turn evaluated by a quarter-length DFT and so on.

The very important computational complexity theorems of Winograd are stated and briefly discussed in Winograd's Short DFT Algorithms. The specific details and evaluations of the Cooley-Tukey FFT and Split-Radix FFT are given in The Cooley-Tukey Fast Fourier Transform Algorithm (Chapter 9), and PFA and WFTA are covered in The Prime Factor and Winograd Fourier Transform Algorithms. A short discussion of high speed convolution is given in Convolution Algorithms, both for its own importance, and its theoretical connection to the DFT. We also present the chirp, Goertzel, QFT, NTT, SR-FFT, Approx FFT, Autogen, and programs to implement some of these.

Ivan Selesnick gives a short introduction in Winograd's Short DFT Algorithms of using Winograd's techniques to give a highly structured development of short prime length FFTs and describes a program that will automatically write these programs. Markus Paschal presents his

“Algebraic Signal Processing” in DFT and FFT: An algebraic viewfast Fourier transform (FFT) is a commonly used algorithmic digital signal processing areas, such as imaging applications and communication systems. Image processing requires computation sizes as high as 222, whereas communication systems apply several computation lengths simultaneously, such as the sizes of 128–2048 in 3GPP long-term evolution (LTE) systems. Consequently, long and variable-size FFT applications are becoming popular. FFT processors can be categorized into two architectures: pipeline- and memory-based architectures. Memory-based FFTs pass the data multiple times through a single butterfly processing element (PE) or set of PEs, with several memory banks to hold intermediate results, processing the data recursively regardless of computation length.

Memory-based FFTs achieve better hardware efficiency, compared with pipeline ones. Nevertheless, the throughput of memory-based FFTs is usually restricted by the butterfly radix and concurrent data access contentions. A fast Fourier transform (FFT) is an algorithm that computes the discrete Fourier transform (DFT) of a sequence, or its inverse (IDFT). Fourier analysis converts a signal from its original domain (often time or space) to a representation in the frequency domain and vice versa. The DFT is obtained by decomposing a sequence of values into components of different frequencies. This operation is useful in many fields, but computing it directly from the definition is often too slow to be practical.

An FFT rapidly computes such transformations by factorizing the DFT matrix into a product of sparse (mostly zero) factors. As a result, it manages to reduce the complexity of computing the DFT from $O(N^2)$, which arises if one simply applies the definition of DFT, to $O(N \log N)$, where N is the data size. The difference in speed can be enormous, especially for long data sets where N may be in the thousands or millions. In the presence of round-off error, many FFT algorithms are much more accurate than evaluating the DFT definition directly or indirectly. There are many different FFT algorithms based on a wide range of published theories, from simple complex-number arithmetic to group theory and number theory.

Fast Fourier transforms are widely used for applications in engineering, music, science, and mathematics. The basic ideas were popularized in 1965, but some algorithms had been derived as early as 1805. In 1994, Gilbert Strang described the FFT as "the most important numerical

algorithm of our lifetime", and it was included in Top 10 Algorithms of 20th Century by the IEEE magazine Computing in Science & Engineering.

The best-known FFT algorithms depend upon the factorization of N , but there are FFTs with $O(N \log N)$ complexity for all N , even for prime N . Many FFT algorithms only depend on the fact that $\{e^{-2\pi i/N}\}$ is an N -th primitive root of unity, and thus can be applied to analogous transforms over any finite field, such as number-theoretic transforms. Since the inverse DFT is the same as the DFT, but with the opposite sign in the exponent and a $1/N$ factor, any FFT algorithm can easily be adapted for it.

4.2 HISTORY OF FFT

The development of fast algorithms for DFT can be traced to Carl Friedrich Gauss's unpublished work in 1805 when he needed it to interpolate the orbit of asteroids Pallas and Juno from sample observations. His method was very similar to the one published in 1965 by James Cooley and John Tukey, who are generally credited for the invention of the modern generic FFT algorithm. While Gauss's work predated even Joseph Fourier's results in 1822, he did not analyze the computation time and eventually used other methods to achieve his goal.

Between 1805 and 1965, some versions of FFT were published by other authors. Frank Yates in 1932 published his version called interaction algorithm, which provided efficient computation of Hadamard and Walsh transforms. Yates' algorithm is still used in the field of statistical design and analysis of experiments. In 1942, G. C. Danielson and Cornelius Lanczos published their version to compute DFT for x-ray crystallography, a field where calculation of Fourier transforms presented a formidable bottleneck. While many methods in the past had focused on reducing the constant factor for $O(N^2)$ computation by taking advantage of "symmetries", Danielson and Lanczos realized that one could use the "periodicity" and apply a "doubling trick" to get $O(N \log N)$ runtime.

James Cooley and John Tukey published a more general version of FFT in 1965 that is applicable when N is composite and not necessarily a power of 2. Tukey came up with the idea during a meeting of President Kennedy's Science Advisory Committee where a discussion topic involved detecting nuclear tests by the Soviet Union by setting up sensors to surround the country

from outside. To analyze the output of these sensors, an FFT algorithm would be needed. In discussion with Tukey, Richard Garwin recognized the general applicability of the algorithm not just to national security problems, but also to a wide range of problems including one of immediate interest to him, determining the periodicities of the spin orientations in a 3-D crystal of Helium-3. Garwin gave Tukey's idea to Cooley (both worked at IBM's Watson labs) for implementation.[Cooley and Tukey published the paper in a relatively short time of six months. As Tukey did not work at IBM, the patentability of the idea was doubted and the algorithm went into the public domain, which, through the computing revolution of the next decade, made FFT one of the indispensable algorithms in digital signal processing.

Let $x_0, \dots, x_{N-1}$ be complex numbers. The DFT is defined by the formula

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N} \quad k=0, \dots, N-1,$$

where $e^{i2\pi/N}$ is a primitive Nth root of 1.

Evaluating this definition directly requires $O(N^2)$ operations: there are N outputs X_k , and each output requires a sum of N terms. An FFT is any method to compute the same results in $O(N \log N)$ operations. All known FFT algorithms require $\Theta(N \log N)$ operations, although there is no known proof that a lower complexity score is impossible. To illustrate the savings of an FFT, consider the count of complex multiplications and additions for $N=4096$ data points. Evaluating the DFT's sums directly involves N^2 complex multiplications and $N(N-1)$ complex additions, of which $O(N)$ operations can be saved by eliminating trivial operations such as multiplications by 1, leaving about 30 million operations.

On the other hand, the radix-2 Cooley–Tukey algorithm, for N a power of 2, can compute the same result with only $(N/2)\log_2(N)$ complex multiplications (again, ignoring simplifications of multiplications by 1 and similar) and $N \log_2(N)$ complex additions, in total about 30,000 operations - a thousand times less than with direct evaluation. In practice, actual performance on modern computers is usually dominated by factors other than the speed of arithmetic operations and the analysis is a complicated subject (for example, see Frigo & Johnson, 2005), but the overall

improvement from $O(N^2)$ to $O(N \log N)$ remains.

The Fourier transform (FT) of the function $f(x)$ is the function $F(\omega)$, where: $F(\omega) = \int_{-\infty}^{\infty} f(x)e^{-i\omega x} dx$ and the inverse Fourier transform is $f(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} F(\omega)e^{i\omega x} d\omega$. Recall that $i = \sqrt{-1}$ and $ei\theta = \cos \theta + i\sin \theta$. Think of it as a transformation into a different set of basis functions. The Fourier transform uses complex exponentials (sinusoids) of various frequencies as its basis functions. (Other transforms, such as Z, Laplace, Cosine, Wavelet, and Hartley, use different basis functions).

A Fourier transform pair is often written $f(x) \leftrightarrow F(\omega)$, or $F(f(x)) = F(\omega)$ where F is the Fourier transform operator. If $f(x)$ is thought of as, a signal (i.e. input data) then we call $F(\omega)$ the signal's spectrum. If f is thought of as the impulse response of a filter (which operates on input data to produce output data) then we call F the filter's frequency response. (Occasionally the line between what's signal and what's filter becomes blurry).

Example of a Fourier Transform

Suppose we want to create a filter that eliminates high frequencies but retains low frequencies (this is very useful in antialiasing). In signal processing terminology, this is called an ideal low pass filter. So we'll specify a box-shaped frequency response with cutoff frequency ω_c : $F(\omega) = 1 \text{ } |\omega| \leq \omega_c \text{ } 0 \text{ } |\omega| > \omega_c$. What is its impulse response? We know that the impulse response is the inverse Fourier transform of the frequency response, so taking off our signal processing hat and putting on our mathematics hat, all we need to do is evaluate: $f(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} F(\omega)e^{i\omega x} d\omega$ for this particular $F(\omega)$: $f(x) = \frac{1}{2\pi} \int_{-\omega_c}^{\omega_c} e^{i\omega x} d\omega = \frac{1}{2\pi} e^{i\omega x} \frac{1}{ix} \Big|_{-\omega_c}^{\omega_c} = \frac{1}{\pi x} \frac{e^{i\omega_c x} - e^{-i\omega_c x}}{2i} = \frac{\sin \omega_c x}{\pi x}$ since $\sin \theta = \frac{e^{i\theta} - e^{-i\theta}}{2i}$ $\omega_c \pi \text{ sinc}(\omega_c \pi x)$ where $\text{sinc}(x) = \sin(\pi x)/(\pi x)$. For antialiasing with unit-spaced samples, you want the cutoff frequency to equal the Nyquist frequency, so $\omega_c = \pi$. Fourier Transform Properties Rather than write "the Fourier transform of an X function is a Y function", we write the shorthand: $X \leftrightarrow Y$. If z is a complex number and $z = x + iy$ where x and y are its real and imaginary parts, then the complex conjugate of z is $z^* = x - iy$. A function $f(u)$ is even if $f(u) = f(-u)$, it is odd if $f(u) = -f(-u)$, it is conjugate symmetric if $f(u) = f^*(-u)$, and it is conjugate antisymmetric if $f(u) = -f^*(-u)$.

The Fast Fourier Transform (FFT) Algorithm The FFT is a fast algorithm for computing the DFT. If we take the 2-point DFT and 4-point DFT and generalize them to 8-point, 16-point... 2^r -point, we get the FFT algorithm. To compute the DFT of an N -point sequence using equation (1) would take $O(N^2)$ multiplies and adds. The FFT algorithm computes the DFT using $O(N \log N)$ multiplies and adds. There are many variants of the FFT algorithm. We will discuss one of them, the “decimation in-time” FFT algorithm for sequences whose length is a power of two ($N = 2^r$ for some integer r). Below is a diagram of an 8-point FFT, where $W = W_8 = e^{-i\pi/4} = (1 - i)/\sqrt{2}$

General FFT Algorithm for $N = 2^r$.

The previously diagrammed algorithm for the 8-point FFT is easily generalized to any power of two. The input array is bit-reversed, and the butterfly coefficients can be seen to have exponents in arithmetic sequence modulo N . For example, for $N = 8$, the butterfly coefficients on the last stage in the diagram are $W_0, W_1, W_2, W_3, W_4, W_5, W_6, W_7$. That is, powers of W in sequence. The coefficients in the previous stage have exponents $0, 2, 4, 6, 0, 2, 4, 6$, which is equivalent to the sequence $0, 2, 4, 6, 8, 10, 12, 14$ modulo 8.

And the exponents in the first stage are $1, -1, 1, -1, 1, -1, 1, -1$, which is equivalent to W raised to the powers $0, 4, 0, 4, 0, 4, 0, 4$, and this is equivalent to the exponent sequence $0, 4, 8, 12, 16, 20, 24, 28$ when taken modulo 8. The width of the butterflies (the height of the “X’s” in the diagram) can be seen to be 1, 2, 4, ... in successive stages, and the butterflies are seen to be isolated in the first stage (groups of 1), then clustered into overlapping groups of 2 in the second stage, groups of 4 in the 3rd stage, etc. The generalization to other powers of two should be evident from the diagrams for $N = 4$ and $N = 8$.

The inverse FFT (IFFT) is identical to the FFT, except one exchanges the roles of a and A , the signs of all the exponents of W are negated, and there is a division by N at the end. Note that the fast way to compute $\text{mod}(j, N)$ in the C programming language, for N a power of two, is with bit-wise AND: “ $j \& (N-1)$ ”. This is faster than “ $j \% N$ ”, and it works for positive or negative j , while the latter does not.

4.3 HOW MUCH FASTER IS THE FFT?

To compute the DFT of an N -point sequence using the definition, $A_k = \sum_{n=0}^{N-1} W_N^{kn} x_n$, would require N^2 complex multiplies and adds, which works out to $4N^2$ real multiplies and $4N^2$ real adds (you can easily check this, using the definition of complex multiplication). The basic computational step of the FFT algorithm is a butterfly. Each butterfly computes two complex numbers of the form $p + \alpha q$ and $p - \alpha q$, so it requires one complex multiply ($\alpha \cdot q$) and two complex adds. This works out to 4 real multiplies and 6 real adds per butterfly.

4.4 CONFLICT FREE ADDRESS SCHEMES

For concurrent data access from different memory banks become an essential problem. A parity bit check method for one or more radix-2 PEs is first introduced in [1]. An in-place strategy from [2] reduces the total memory storage to minimum N . In [3], a mixed radix-4/2 in-place scheme makes the input and output bits symmetric, and then, the conflict-free scheme is extended to a mixed-radix algorithm. In [4], a multiple radix-2 PE scheme is demonstrated to increase the throughput of FFT processors. However, the methods are only suitable for power-of-two point FFTs. In [5], a single- and multiple-PE method for arbitrary radix- b algorithm is discussed. However, it requires memory in every stage. In a multipath delay commutator, (MDC) architecture with high radix is used to replace the complex PE in conventional memory-based FFTs. This method provides low-power dissipation, high data rates, good computational efficiency, and refined length flexibility.

However, none of them provides a detailed conflict-free address scheme. In [6], a generalized conflict-free address scheme with single or multiple radix- 2^q MDC architectures for 2^n -point FFTs is illustrated. However, this scheme does not extend the principle to arbitrary-length FFTs and more general decomposition algorithms. Generalized mixed-radix algorithms that support both traditional 2^n -point and prime-sized FFTs are proposed in [7] and [8]. Although they obtain the memory bank and address rules according to the decomposition algorithms, they do not state the data distribution procedure clearly, especially when prime factor algorithm (PFA) is applied within the FFT. Moreover, they cannot fully support the continuous flow working mode.

A memory-based FFT processor design methodology with a generalized conflict-free address scheme for arbitrary-length FFTs. We unify the conflict-free address schemes of three different FFT lengths, including the single power point (SPP) FFTs, the common non single-power point (NSPP) FFTs, and the NSPP FFTs applied with the PFA, to the same address generation format. The memory bank index and the internal address are all generated by modulo and multiplication operations of the decomposition digits. Moreover, a decomposition algorithm, named highradix–small-butterfly (HRSB), utilizes high-radix PE to reduce the total number of computation cycles and small butterfly units (BUs) to eliminate the complexity of the computation engine. Thus, the FFTs can complete N-point computations within N cycles to support the continuous-flow mode with low complexity. In the previous methods two successive identical symbols produce two slightly different results, because the factorization changes after each transform. To avoid this problem, we apply the same factorization to every data symbol. Furthermore, an efficient index generator, a simplified configurable MDC unit, and a unified Winograd Fourier transform algorithm (WFTA) butterfly core for point-2,3,4,5 DFTs are designed for the prime-factor FFTs. We designed two FFT examples in LTE systems, including a 2048-point FFT unit and a DFT unit with 35 different points (12–1296). The techniques proposed previously can be extended to arbitrary-length FFTs.

4.5 FFT ARCHITECTURE

Not only have several variations of the FFT algorithm been developed after the Cooley-Tukey's publication but also various implementations. Generally, the FFT can be implemented in software, general-purpose digital signal processors, application specific processors or algorithm-specific processors. The implementations with software on general-purpose computer can be found in literature and still being explored in some projects, for instance, the FFTW project in the Laboratory for Computer Science at MIT. Software implementations are not suitable for our target application as the power consumption is too high. Since it is hard to summarize all other implementations, we will concentrate on algorithmic-specific architectures and only give a brief overview on some FFT architectures. General-Purpose Programmable DSP Processors Many commercial programmable DSP processors include the special instructions for the FFT computation. Although the performance varies from one to another, most of them belong to the Harvard architecture from the architecture point of view. A processor with Harvard architecture

has separate busses for data and control. A typical programmable DSP processor has on chip data and program memory, address generator, program control, MAC, ALU, and I/O interfaces, as illustrated.

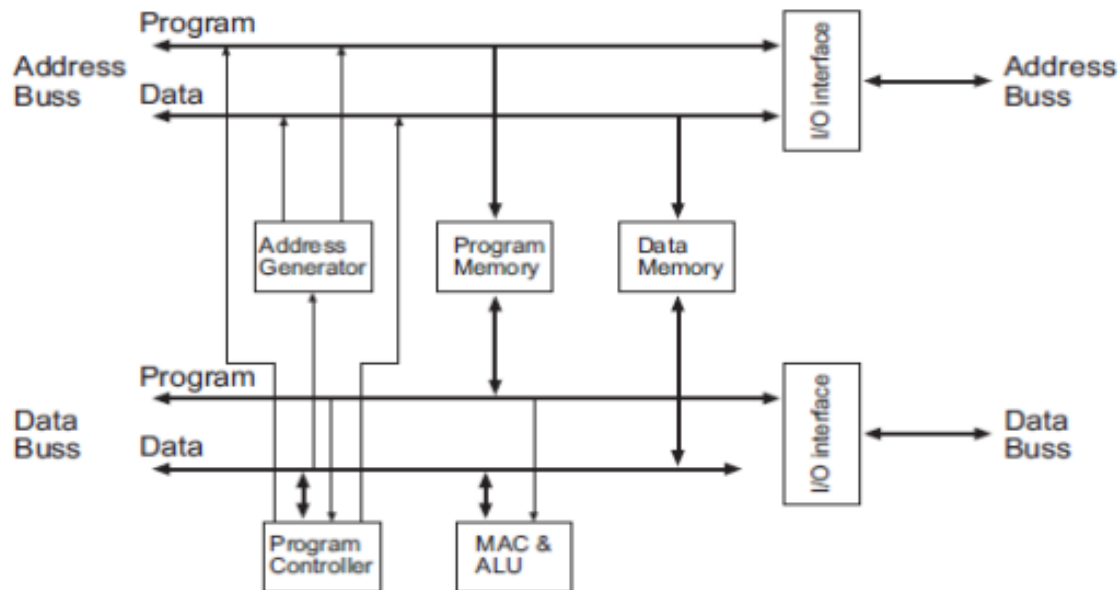


Fig 4.1: General-purpose programmable DSP processor

The computation of FFT with general-purpose DSP processor does not differ too much from the software computation of FFT in a general-purpose computer. To compute the FFT with a general-purpose DSP processor requires three steps: first the data input, then the FFT/IFFT computation, and finally the data output. In some DSP processors, for instance TI's TMS320C3x, bit-reverse addressing is available to accelerate the unscrambling for the data output. Typical FFT/IFFT execution times are about 1 ms. which is far from the implementation using more specialized implementations. The implementation with general-purpose programmable DSP processor is therefore not applicable due to the throughput requirement.

4.6 PROGRAMMABLE FFT SPECIFIC PROCESSORS

Several programmable FFT processors have been developed for the FFT/IFFT computations. These processors are 5 to 10 times faster than the general-purpose programmable DSP processors. The programmable FFT-specific processors have specific butterflies and at least one complex multiplier. The butterfly is usually radix-2 or radix-4. There is often an on-chip coefficient ROM, which stores sinus and cosine coefficients. This type of programmable FFT-specific processors are often provided with windowing functions in either time or frequency domain. The Zarlink's (former Plessey) PDSP16515A processor performs decimation in time, radix 4, forward or inverse Fast Fourier Transforms. Data are loaded into an internal workspace RAM in normal sequential order, processed, and then readout in correct order. The processor has two internal workspace RAMs, one output buffer, and one coefficient ROM

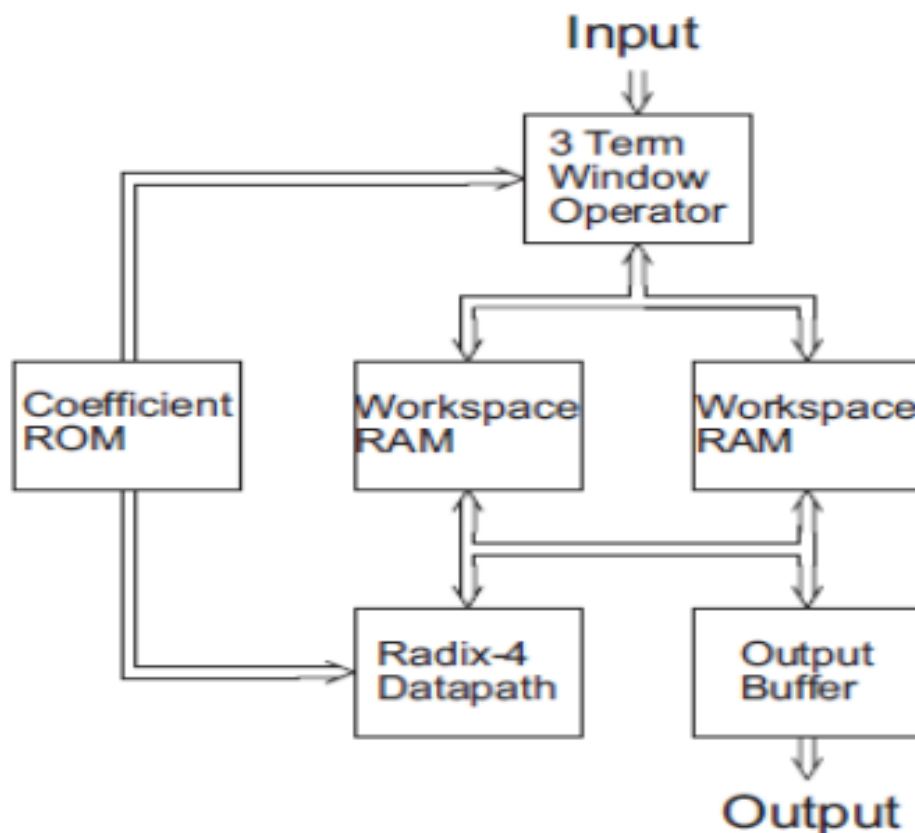


Fig 4.2: FFT-specific processor PDSP16515A

Although the PDSP1615A processor accelerates the FFT computation, it is still hard to meet the throughput requirement with a single processor due to the slow I/O. The processor requires 98 ms. to perform 1024-point FFT with a system clock of 40 MHz. Using multiple processor configuration can achieve a higher throughput, but the power consumption is then substantially higher. A recent released FFT specific processor from DoubleBW systems B. V. has higher throughput (100 Msamples/s) but consumes 8 W at 3.3 V.

4.7 ALGORITHM-SPECIFIC PROCESSORS

Non-programmable algorithm-specific processors can also be designed for the computation of FFT algorithms. The processors are designed mostly for fixedlength FFTs. The architecture of an algorithmic-specific FFT processor is therefore optimized with respect to memory structure, control units, and processing elements. There are mainly three types of algorithm-specific processors: fully parallel FFT processors, column FFT processors, and pipelined FFT processors. All three types of algorithm-specific processors represent different mapping of the signal-flow graph for FFT to hardware structures. The hardware structure in a fully parallel FFT processor is an isomorphic mapping of the signal-flow graph. For example, the signal-flow graph for an 8-point FFT algorithm is shown. The 8-point fully parallel FFT processor requires 24 complex adders and 5 complex multipliers. The hardware requirement is excessive, and, hence, is not power efficient.

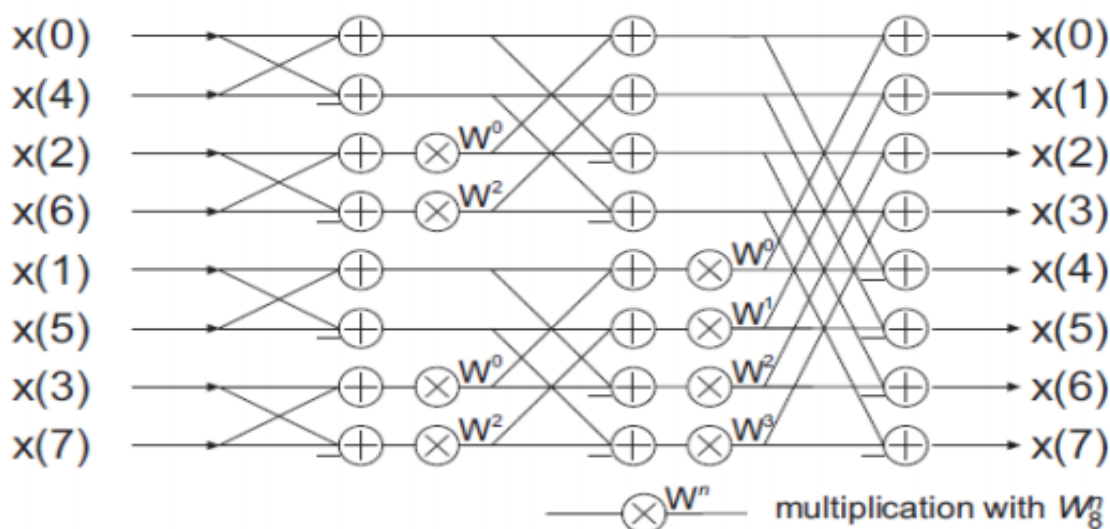


Fig 4.3:8-point fully parallel FFT processor

To reduce the hardware complexity, a column or a pipelined FFT processor can be used. A set of process elements in a column FFT processor compute one stage at a time. The results are fed back to the same set of process elements to compute the next stage. For the long transform length, the routing for the processing elements is complex and difficult. For a pipelined FFT processor, each stage has its own set of processing elements. All the stages are computed as soon as data are available. pipelined FFT processors have features like simplicity, modularity and high throughput. These features are important for real-time, in-place applications where the input data often arrive in a natural sequential order. We therefore select the pipeline architecture for our FFT processor implementation.

The most common groups of the pipelined FFT architecture are

- Radix-2 multipath delay commutator (R2MDC)
- Radix-2 single-path delay feedback (R2SDC)
- Radix-4 multipath delay commutator (R4MDC)
- Radix-4 single-path delay commutator (R4SDC)
- Radix-4 single-path delay feedback (R4SDF)
- Radix-2 2 single-path delay commutator (R22 SDC)

We will discuss these pipeline architectures in more detail.

4.7.1 RADIX-2 MULTIPATH DELAY COMMUTATOR

The Radix-2 Multipath Delay Commutator (R2MDC) architecture is the most straightforward approach to implement the radix-2 FFT algorithm using a pipeline architecture [82]. An 8-point R2MDC FFT is shown

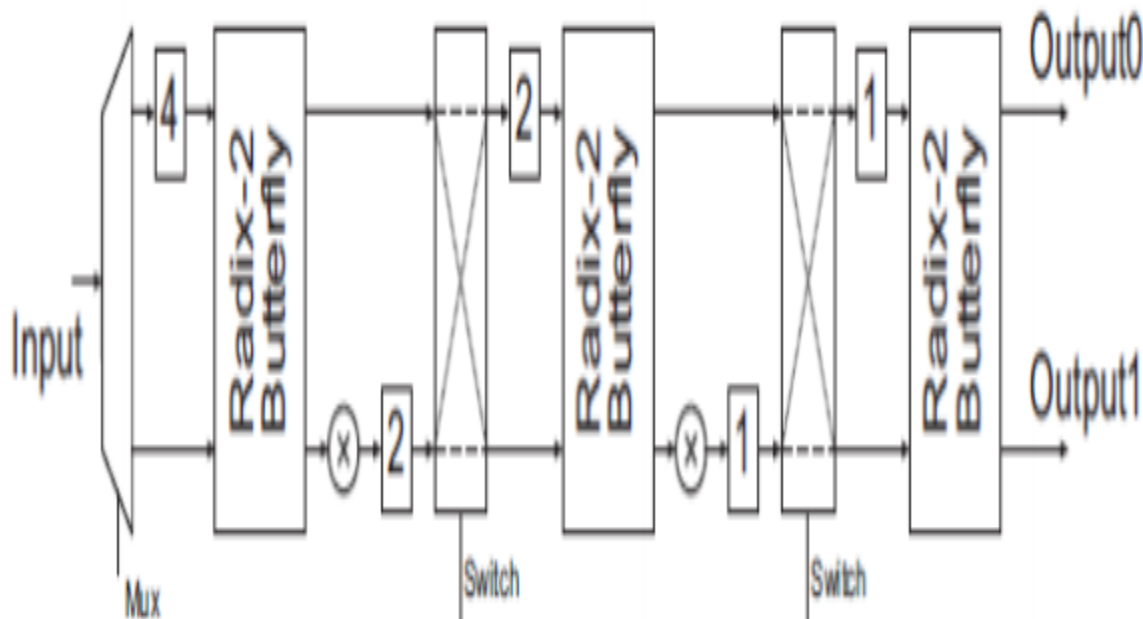


Fig 4.4: 8-point R2MDC FFT

When a new frame arrives, the first four input data are multiplexed to the top-left delay elements in the figure and the next four input data directly to the butterfly. In this way the first input data is delayed by four samples and arrives to the butterfly simultaneously with the fourth input sample. This completes the start-up of the first stage of the pipeline. The outputs from the first stage butterfly and the multiplier are then fed into the multipath delay commutator between stage 1 and stage 2. There are two paths (multipath) with delay elements and one switch (commutator). The multipath delay commutator alleviates the data dependency problem. The first and second outputs from the upper side of the butterfly are fed into the two upper delay elements. After this, the switch changes and the third and fourth outputs from the upper output of the first butterfly are sent directly to the butterfly at stage 2. However, the first and second outputs from the multiplier at the first stage are now delayed by the upper delay elements, which make the first and second outputs from the multiplier of the first stage arrive together with the fifth and sixth outputs from the top.

The butterfly and the multiplier are idle half the time to wait for the new inputs. Hence the utilization of the butterfly and the multiplier is 50%. The total number of delay elements is $4 + 2$

$+ 2 + 1 + 1 = 10$ for the 8-point FFT. The total number of delay elements for an N-point FFT can be derived in similar way and is $N/2 + N/2 + N/4 + \dots + 2$, i.e., $3N/2 - 2$. Each stage (except the last one) has one multiplier and the number of multipliers is $\log_2(N) - 1$.

4.7.2 Radix-2 Single-Path Delay Feedback

Herbert L. Groginsky and George A. Works introduced a feedback mechanism in order to minimize the number of delay elements. In the proposed architecture one half of outputs from each stage are fed back to the input data buffer when the input data are directly sent to the butterfly. This architecture is called Radix-2 Single-path Delay Feedback (R2SDF).

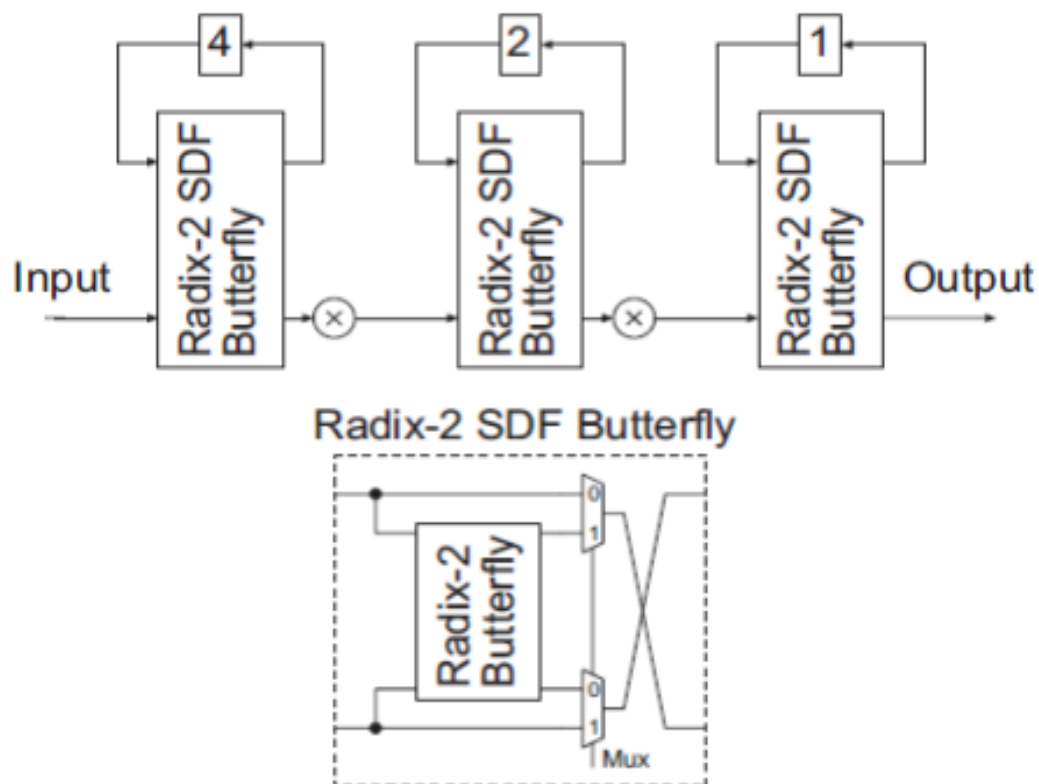


Fig 4.5: Radix-2 Single-path Delay Feedback

4.7.3 8-POINT DIF R2SDC FFT

The delay elements at the first stage save four input samples before the computation starts. During the execution, they store one output from the butterfly of the first stage and one output is immediately transferred to the next stage. Thus, in the new interim half frame when the delay

elements are filled with fresh input sample, the results of the previous frame are sent to the next stage. The butterfly is provided with a feedback loop. The modified butterfly is shown in the right side of Fig. 4.5. When the mux is 0, the butterfly is idle and data passes by. When the mux is 1, the butterfly processes the incoming samples. Because of the feedback mechanism, we reduce the requirement of delay elements from $3N/2$ to $N - 1$ ($N/2 + N/4 + \dots + 1$) which is minimal. The number of multiplier is exact the same as R2MDC FFT architecture, i.e., $\log_2(N) - 1$. The utilization of multiplier and butterflies remains the same, namely 50%.

4.7.4 RADIX-4 MULTIPATH DELAY COMMUTATOR

This architecture is similar to R2MDC. Input data are separated by a 4-to-1 multiplexer and $3N/2$ delay elements at the first stage. A 4-path delay commutator is used between two stages. Computation is taking place only when the last 1/4 part of data is multiplexed to the butterfly. The utilization of the butterflies and the multipliers is 25%. The length of the FFT has to be $4n$. A length-64 DIF Radix-4 Multipath Delay Commutator (R4MDC) FFT is shown

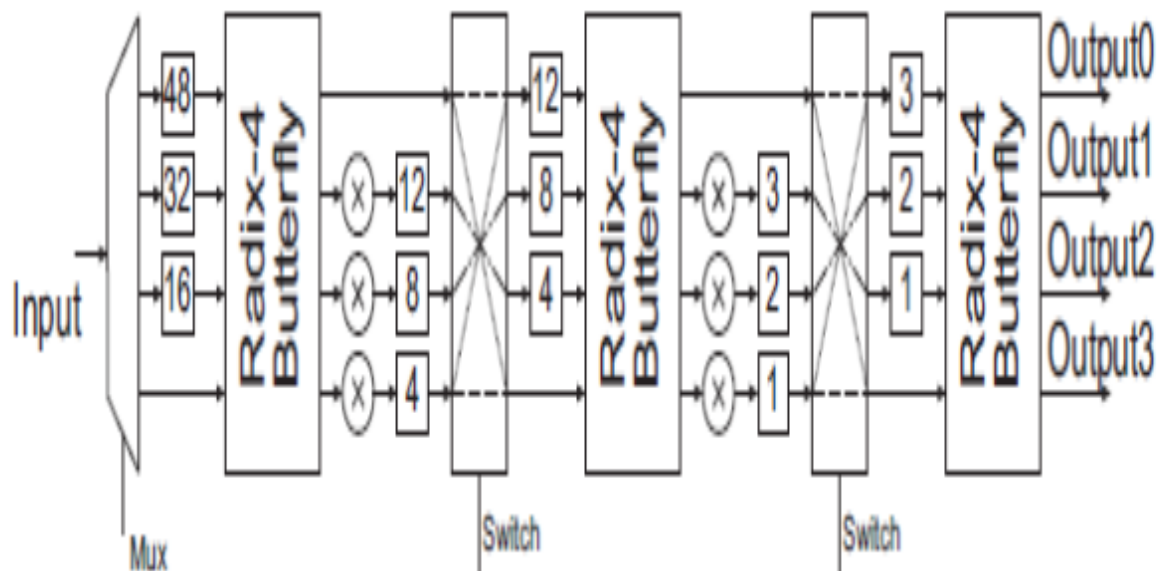


Fig 4.6:64-point DIF R4MDC FFT

Each stage (except the last stage) has 3 multipliers and the R4MDC FFT requires in total $3 \cdot (\log_4(N)-1)$ multipliers for an N -point FFT which is more than the R2MDC or R2SDF. Moreover, the memory requirement is $5N/2-4$, which is the largest among the three discussed architectures. From the view of hardware and utilization, it is not a good structure.

4.7.5 RADIX-4 SINGLE-PATH DELAY COMMUTATOR

To increase the utilization of the butterflies, G. Bi and E. V. Jones [84] proposed a simplified radix-4 butterfly. In the simplified radix-4 butterfly, only one output is produced in comparison with 4 in the conventional butterfly. To provide the same four outputs, the butterfly works four times instead of just one. Due to this modification the butterfly has a utilization of 4.25 % or 100%. To accommodate this change we must provide the same four data at four different times to the butterfly. A few more delay elements are required with this architecture. Furthermore, the simplified butterfly needs additional control signals, and so do the commutators. The number of multipliers is $\log_4(N)-1$, which is less than the R4MDC FFT architecture. The utilization of the multiplier is 75% due to the fact that at least one-fourth of the data are multiplied with the trivial twiddle factor 1 (no multiplication is needed). The structure of a 16-point DIF Radix-4 Single-Path Delay Commutator (R4SDC) FFT is shown below.

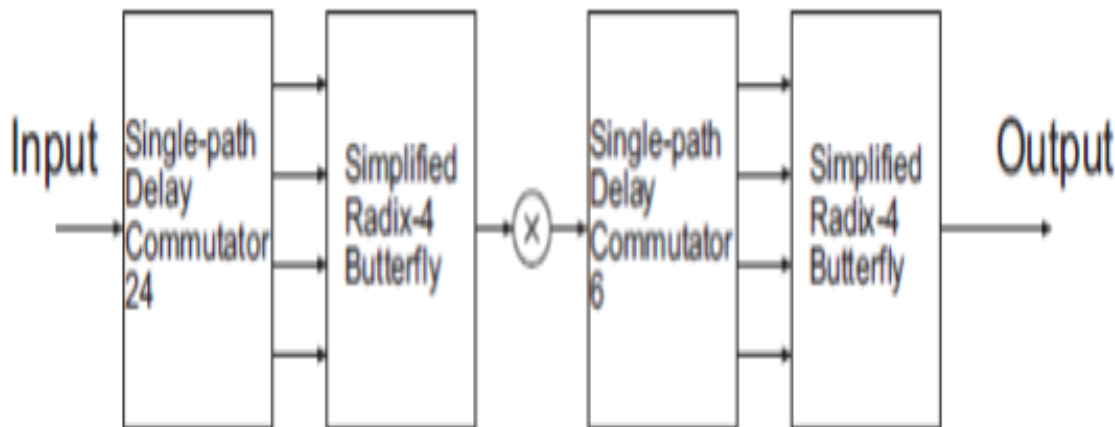


Fig 4.7: 16-point DIF Radix-4 Single-Path Delay Commutator (R4SDC) FFT

The main benefit of this architecture is the utilization improvement for butterflies. The cost for R4SDC is the increase amounts of delay elements.

4.7.6 RADIX-4 SINGLE-PATH DELAY FEEDBACK

Radix-4 single-path delay feedback (R4SDF) [85] [86] is a radix-4 version of R2SDF. Since we use the radix-4 algorithm we can reduce the number of multipliers to $\log_4(N) - 1$ compared to $\log_2(N) - 2$ for R2SDF. But the utilization of the butterflies are reduced to 25%. The radix-4 SDF butterflies also become more complicated than the radix-2 SDF butterflies. A 64-point DIF R4SDF FFT is illustrated

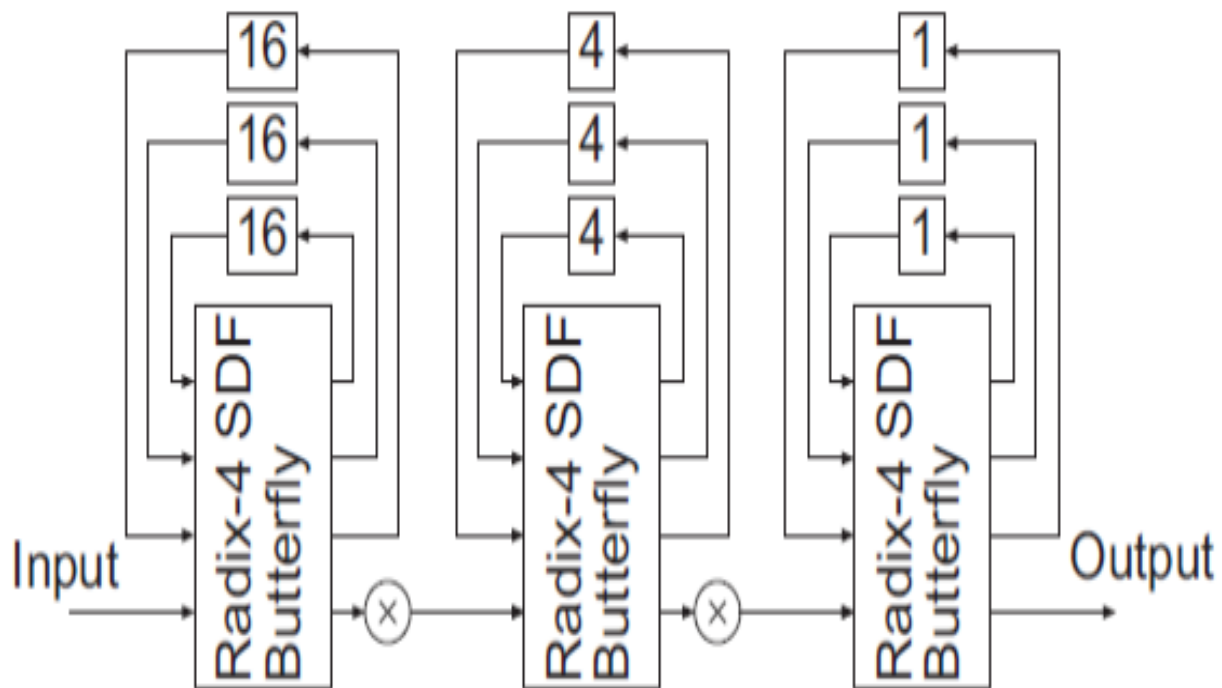


Fig 4.8: 64-point DIF R4SDF FFT.

The radix-4 SDF butterfly is shown in Fig. 4.9. The data are sent to the butterfly for processing when the mux is 1, otherwise the data are shifted into a delay-line with a length of $3N/4$ (first stage).

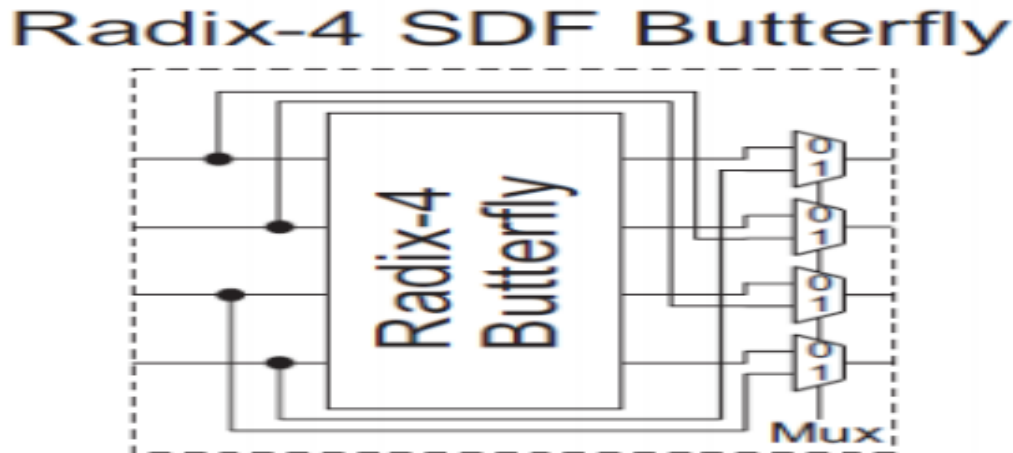


Fig 4.9: radix-4 SDF butterfly

4.7.7 RADIX-2 2 SINGLE-PATH DELAY COMMUTATOR

The Radix-2 2 Single-path Delay Commutator (R22 SDC) architecture uses a modified radix-4 DIF FFT algorithm. It has the same butterfly structure as the radix-2 DIF FFT, but places the multipliers at the same places as for the radix-4 DIF FFT. Basically two kinds of radix-2 SDF butterflies are used to achieve the same output (but not of the same order) as a radix-4 butterfly. By reducing the radix from 4 to 2 we increase utilization of the butterflies from 25% to 50%. We reduce the number of multipliers compared to the conventional radix-2 algorithm. This approach is based on a 4-point DFT

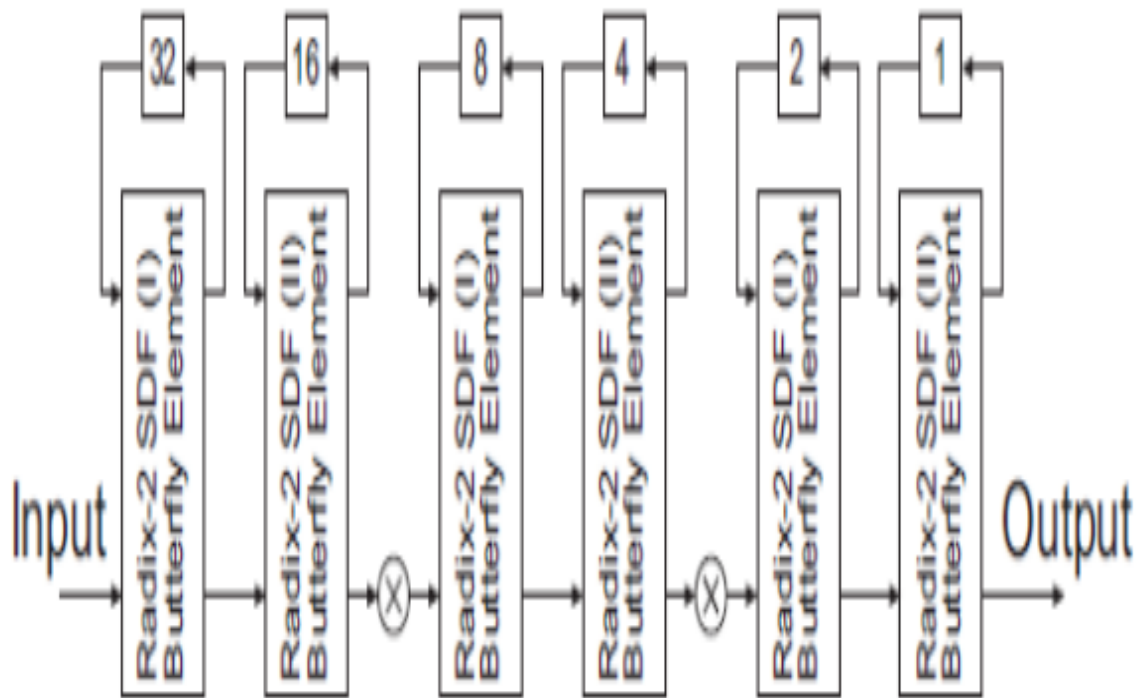


Fig 4.10: Radix-2 2 Single-path Delay Commutator

The outputs are bit-reversed instead of 4-reversed as in a conventional radix-4 algorithm.

CHAPTER 5

PROPOSED SYSTEM ARCHITECTURE

5.1 BLOCK DIAGRAM

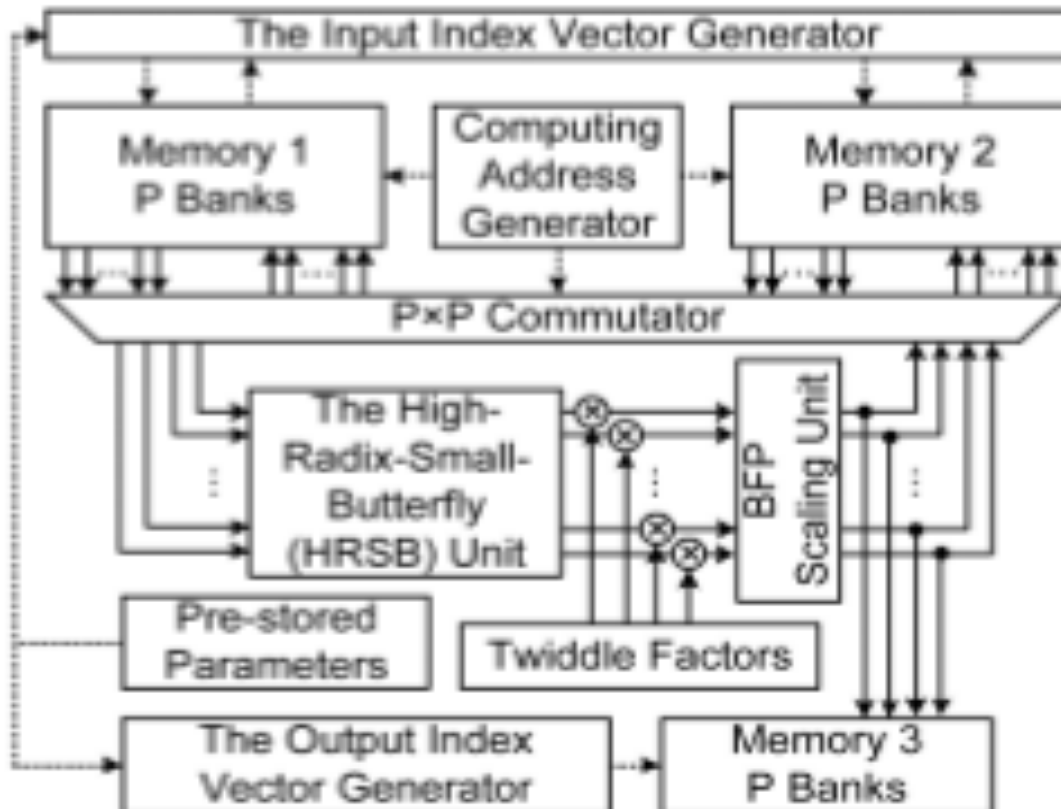


FIG 5.1 BLOCKDIAGRAM OF PROJECT

5.2 DESCRIPTION OF BLOCKDIAGRAM

The entire FFT processor architecture consists of the following main parts: an input and an output index vector generator, a computation address generator, three different memory bank groups, a PE unit applied with the HRSB scheme, some commutators between. The memory and the PE, some prestored twiddle factors, and FFT size parameters, and an exponent scaling unit. The dashed lines represent the control signals while the real lines denote the flowing data. The

input index vector generator distributes the input data to different memory banks without data conflicts, and the output one reorders the output data to a natural sequence.

The computation address generator obtains all the concurrent data of each cycle and stores back the intermediate results. Memory groups 1 and 2 are in a ping-pong mode to hold two continuous data symbols in input sampling, and memory group 3 is used to output the computed data in right order. Three memory groups only exist in NSPP FFTs. For SPP FFTs, as in-place strategy is available, only two memory groups are enough. The HRSB unit is the kernel processing engine. The commutators located between the memories and HRSB unit provide efficient data routing mechanism which is controlled by the computation address generator. The twiddle factors and the FFT size parameters are all prestored in register files, which are used to configure the FFT working modes. The exponent scaling unit controls scaling operations for block floating-point, which can increase the signal-to-quantization noise ratio and reduce the memory storage.

5.3 SCHEMATIC DIAGRAM

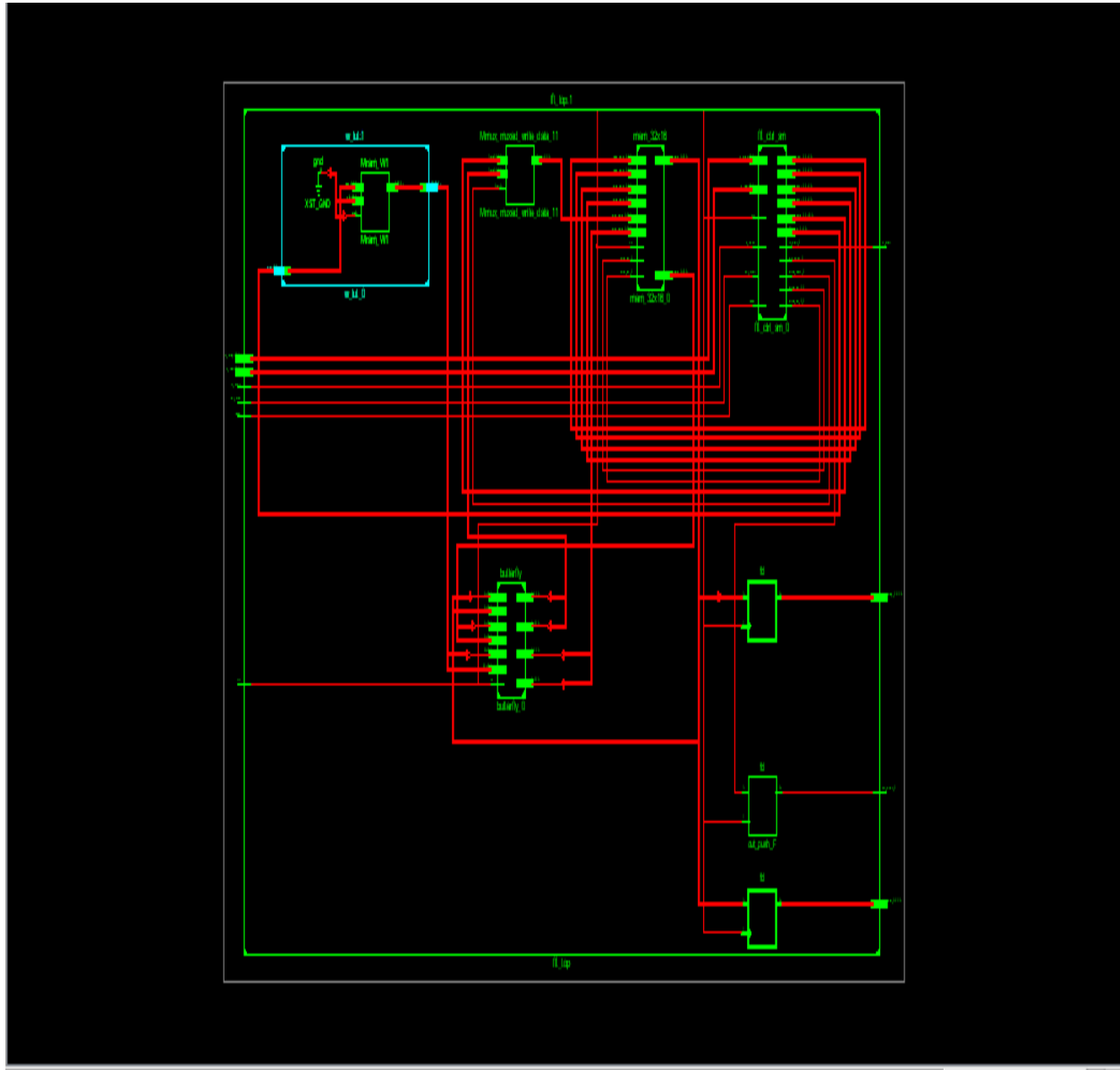


Fig 5.2 :Schematic diagram of fft processor

5.4 FLOW CHART

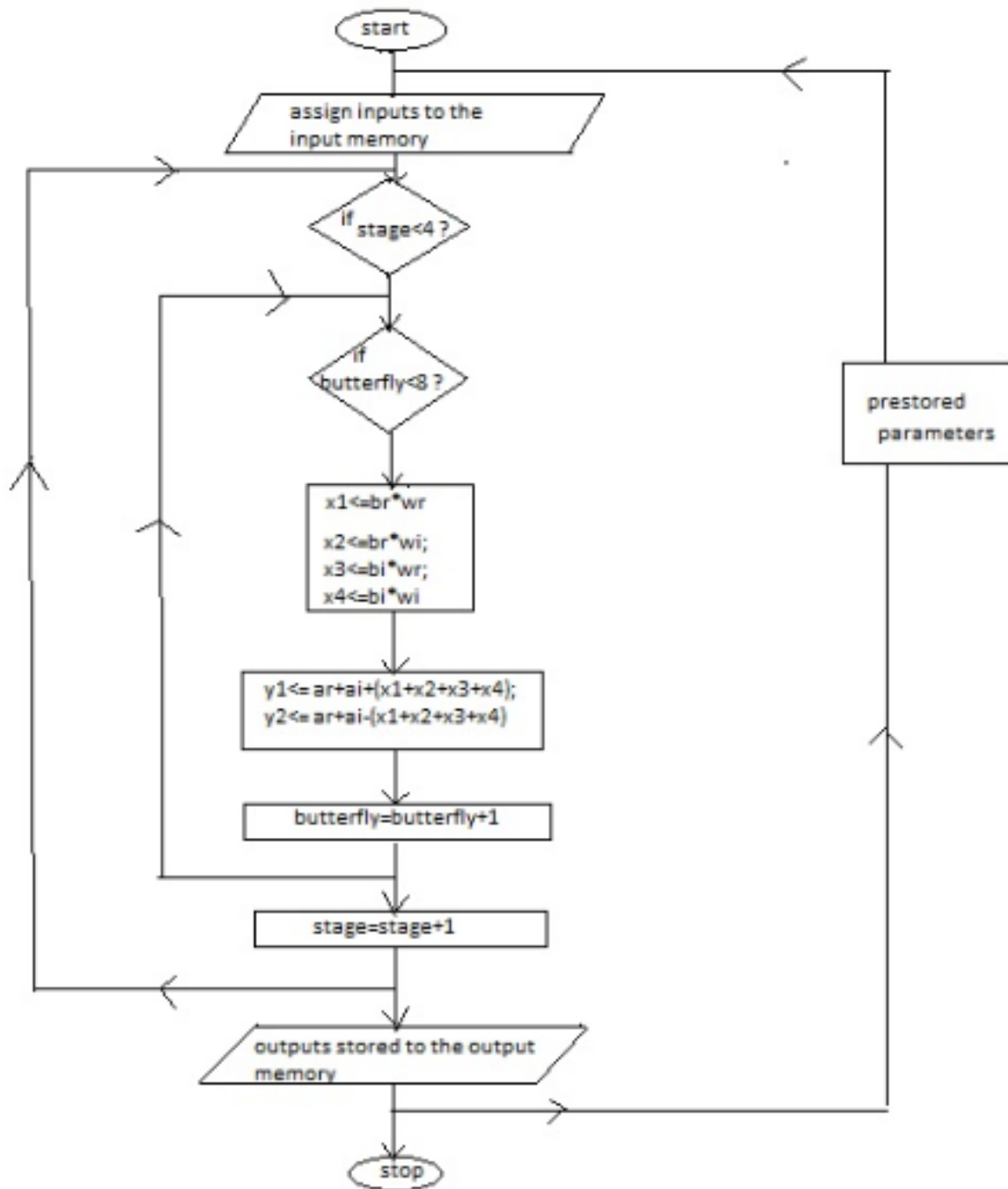


Fig 5.3 :flow chart

5.5 RESULT

The memory based fft processor is designed for 16 point fft and the output waveform are shown in fig 5.4.

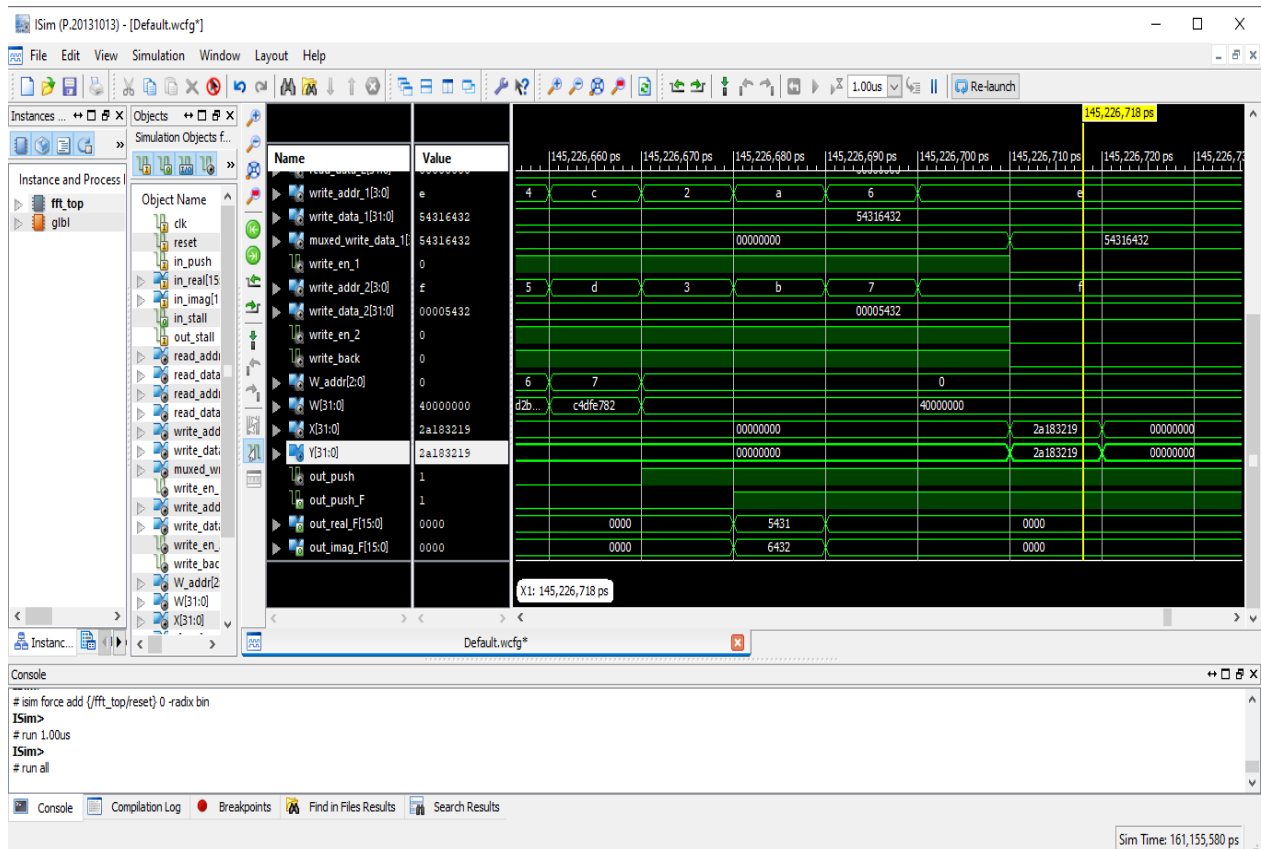


fig 5.4:output waveform of fft processor

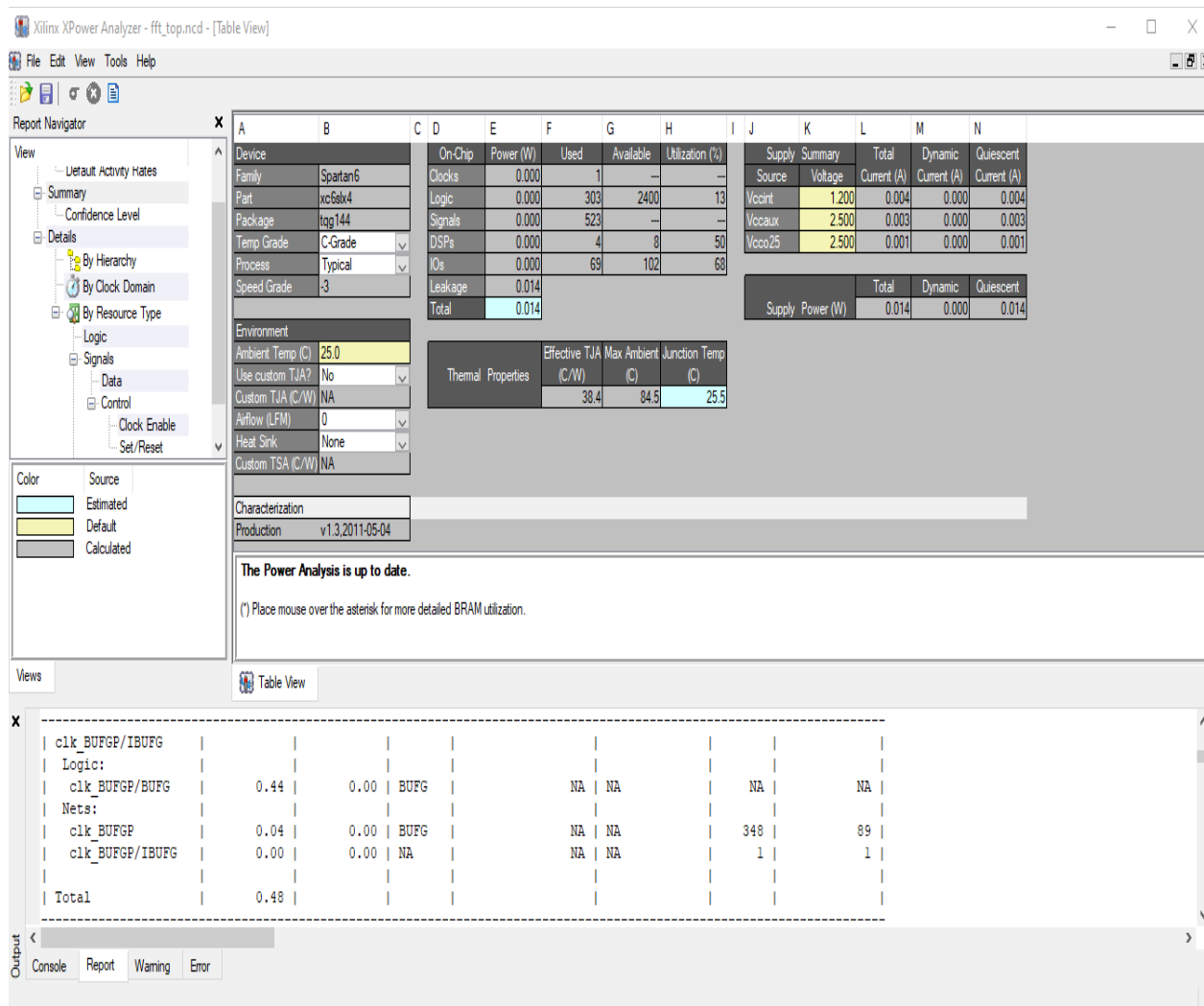


Fig 5.5 : Power Analyzer

ADVANTAGES

- FFT helps in converting the time domain in frequency domain which makes the calculations easier.
- It can convert the discrete data into a continuous data type available at various frequencies.

CONCLUSION

We present memory-based FFT implementations with generalized efficient conflict-free address schemes. Address schemes for different FFT lengths are integrated in this paper to support FFT processing for various systems. The memory bank and address can be generated by modulo and multiplication operations of the decomposition digits. For both SPP and NSPP FFTs, high-radix algorithm and parallel-processing technique can be used to increase the throughput. And the address scheme for FFTs applied with PFA is explored. Moreover, a decomposition method, named HRSB, is designed to suit the high-radix algorithm. Full hardware architectures for the FFTs in LTE systems are illustrated, including the index vector generator, the butterfly engine, and the unified WTFA core. The implementation results and comparisons are also presented

BIBLIOGRAPHY

- [1] C.-L. Yu, K. Irick, C. Chakrabarti, and V. Narayanan, “Multidimensional DFT IP generator for FPGA platforms,” *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 58, no. 4, pp. 755–764, Apr. 2011.
- [2] C.-H. Yang, T.-H. Yu, and D. Markovic, “Power and area minimization of reconfigurable FFT processors: A 3GPP-LTE example,” *IEEE J. Solid-State Circuits*, vol. 47, no. 3, pp. 757–768, Mar. 2012.
- [3] D. Cohen, “Simplified control of FFT hardware,” *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 24, no. 6, pp. 577–579, Dec. 1976.
- [4] M. C. Pease, “Organization of large scale Fourier processors,” *J. ACM*, vol. 16, no. 3, pp. 474–482, Jul. 1969.
- [5] L. G. Johnson, “Conflict free memory addressing for dedicated FFT hardware,” *IEEE Trans. Circuits Syst. II, Analog Digit. Signal Process*, vol. 39, no. 5, pp. 312–316, May 1992.
- [6] B. G. Jo and M. H. Sunwoo, “New continuous-flow mixedradix (CFMR) FFT processor using novel in-place strategy,” *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 52, no. 5, pp. 911–919, May 2005.
- [7] J. Baek and K. Choi, “New address generation scheme for memorybased FFT processor using multiple radix-2 butterflies,” in *Proc. Int. SoC Design Conf.*, vol. 1, Nov. 2008, pp. I-273–I-276.
- [8] D. Reisis and N. Vlassopoulos, “Conflict-free parallel memory accessing techniques for FFT architectures,” *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 55, no. 11, pp. 3438–3447, Dec. 2008.
- [9] K. H. Chen and Y. S. Li, “A multi-radix FFT processor using pipeline in memory-based architecture (PIMA) FOR DVB-T/H systems,” in *Proc. Int. Conf. Mixed Design Integr. Circuits Syst.*, Jun. 2008, pp. 549–553.

- [10] S.-Y. Lee, C.-C. Chen, C.-C. Lee, and C.-J. Cheng, “A low-power VLSI architecture for a shared-memory FFT processor with a mixed-radix algorithm and a simple memory control scheme,” in Proc. IEEE Int. Symp. Circuits Syst., May 2006, pp. 157–160.
- [11] P.-Y. Tsai, T.-H. Lee, and T.-D. Chiueh, “Power-efficient continuousflow memory-based FFT processor for WiMax OFDM mode,” in Proc. Int. Symp. Intel. Signal Process. Commun., Dec. 2006, pp. 622–625.
- [12] P.-Y. Tsai and C.-Y. Lin, “A generalized conflict-free memory addressing scheme for continuous-flow parallel-processing FFT processors with rescheduling,” IEEE Trans. Very Large Scale Integr. (VLSI) Syst., vol. 19, no. 12, pp. 2290–2302, Dec. 2011.
- [13] C.-F. Hsiao, Y. Chen, and C.-Y. Lee, “A generalized mixed-radix algorithm for memory-based FFT processors,” IEEE Trans. Circuits Syst. II, Express Briefs, vol. 57, no. 1, pp. 26–30, Jan. 2010.
- [14] J. Chen, J. Hu, S. Lee, and G. E. Sobelman, “Hardware efficient mixed radix-25/16/9 FFT for LTE systems,” IEEE Trans. Very Large Scale Integr. (VLSI) Syst., vol. 23, no. 2, pp. 221–229, Feb. 2015.
- [15] C. Burrus, “Index mappings for multidimensional formulation of the DFT and convolution,” IEEE Trans. Acoust., Speech, Signal Process., vol. 25, no. 3, pp. 239–242, Jun. 1977.